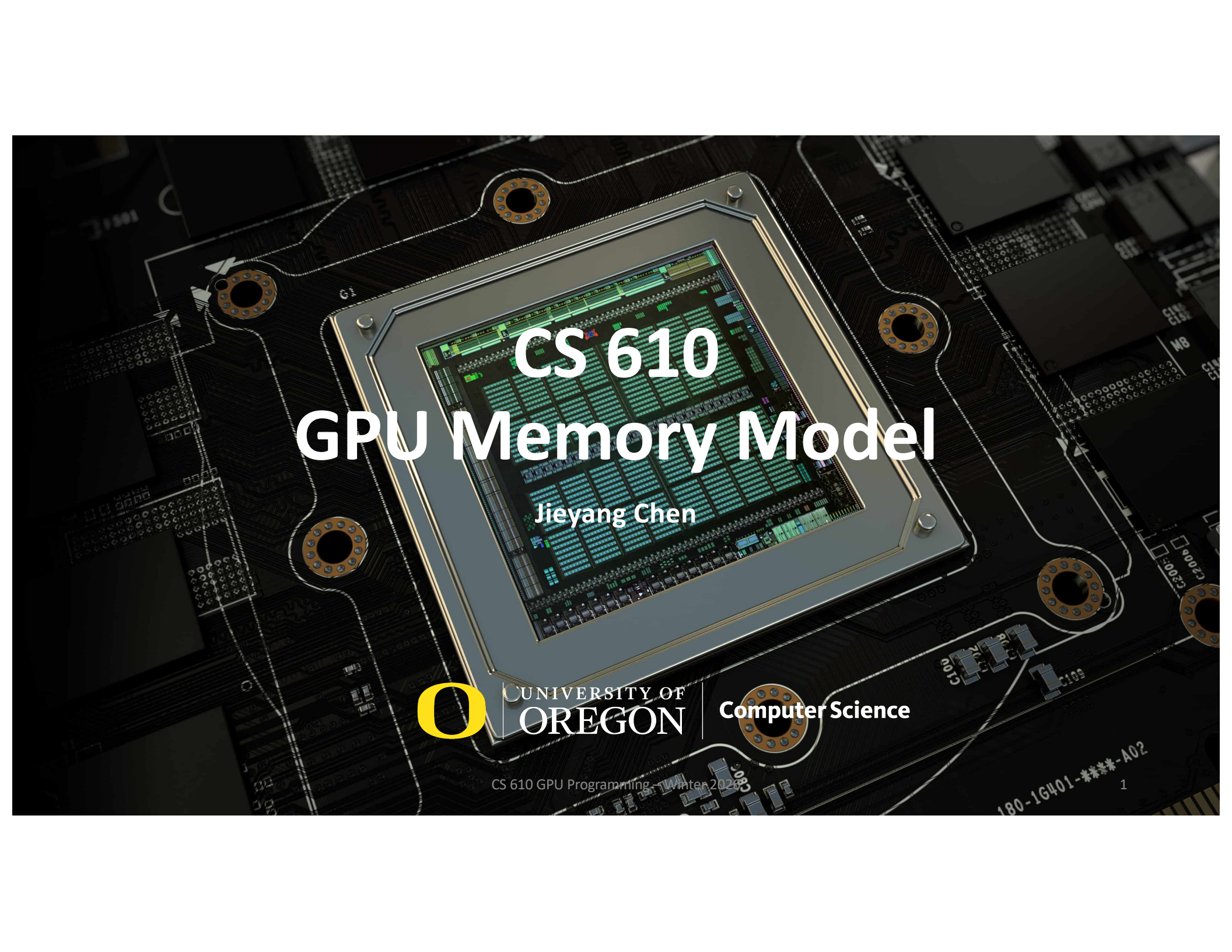




CS 610 GPU Memory Model

Jieyang Chen



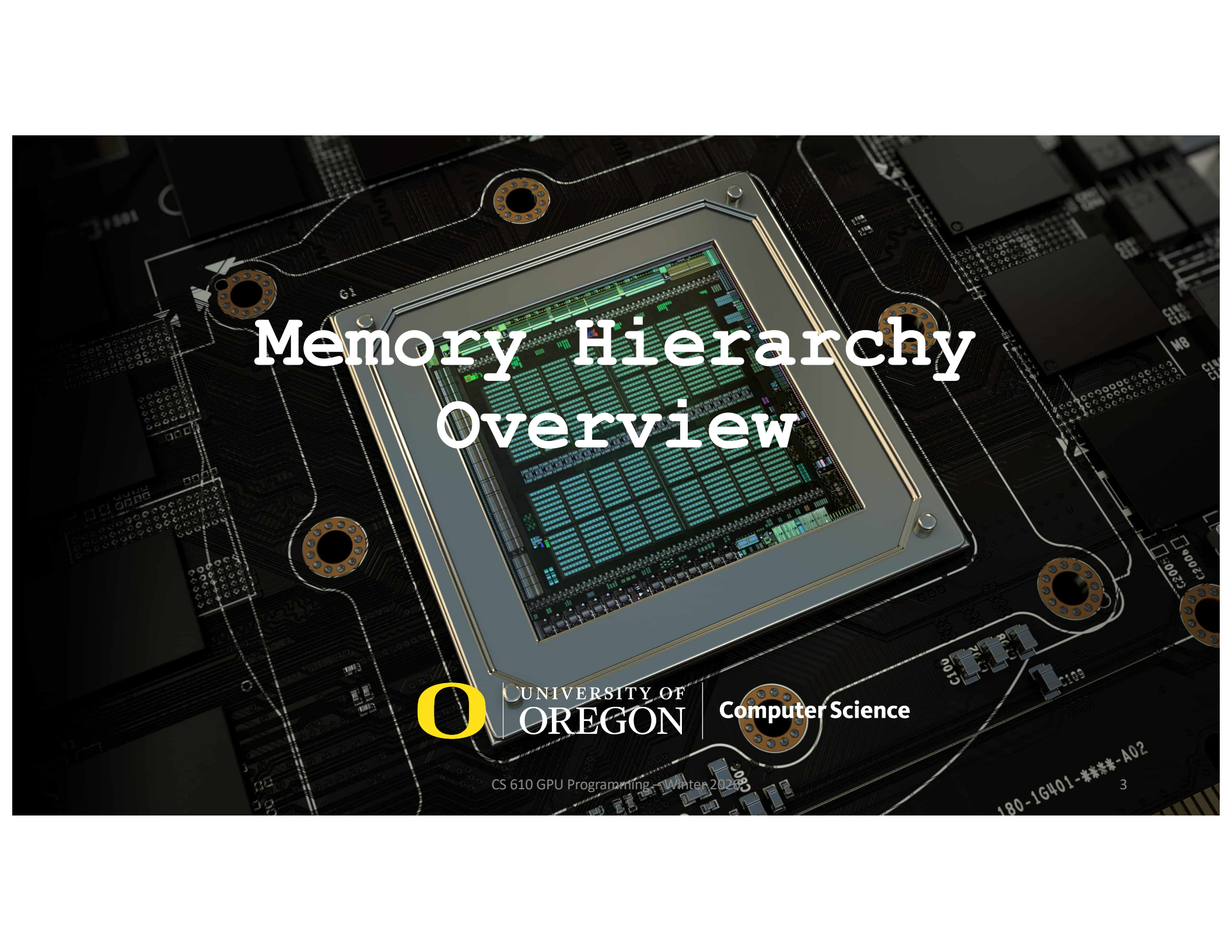
UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

Updates

- Read chapter 4 and 5
- Coding assignment due 1/18
- Choose coding option by 1/21 (on Canvas quiz)



Memory Hierarchy Overview



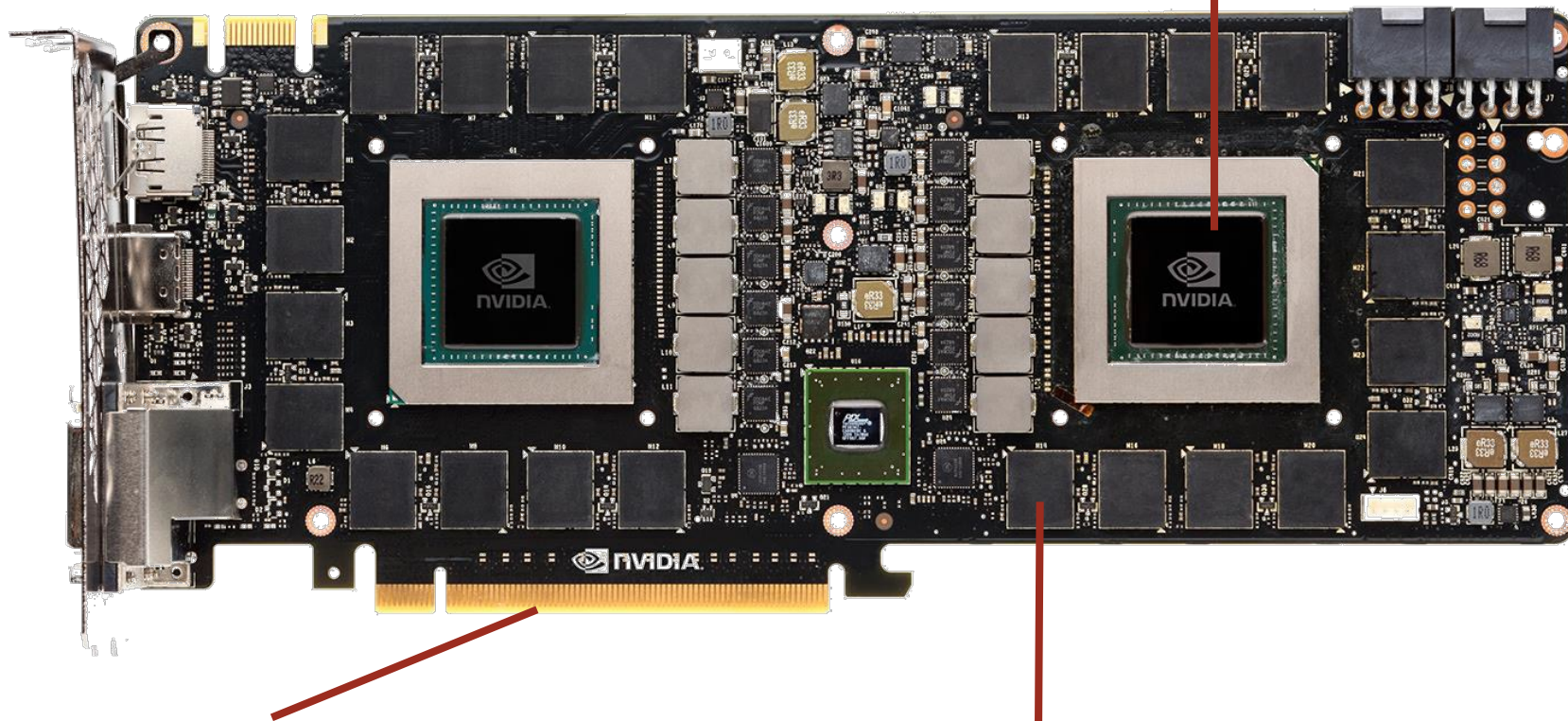
UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

GPU Memory (GTX Titan Z)

Shared Memory, cache and registers



Host Memory (via PCIe)

GPU DRAM Memory



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming – Winter 2026

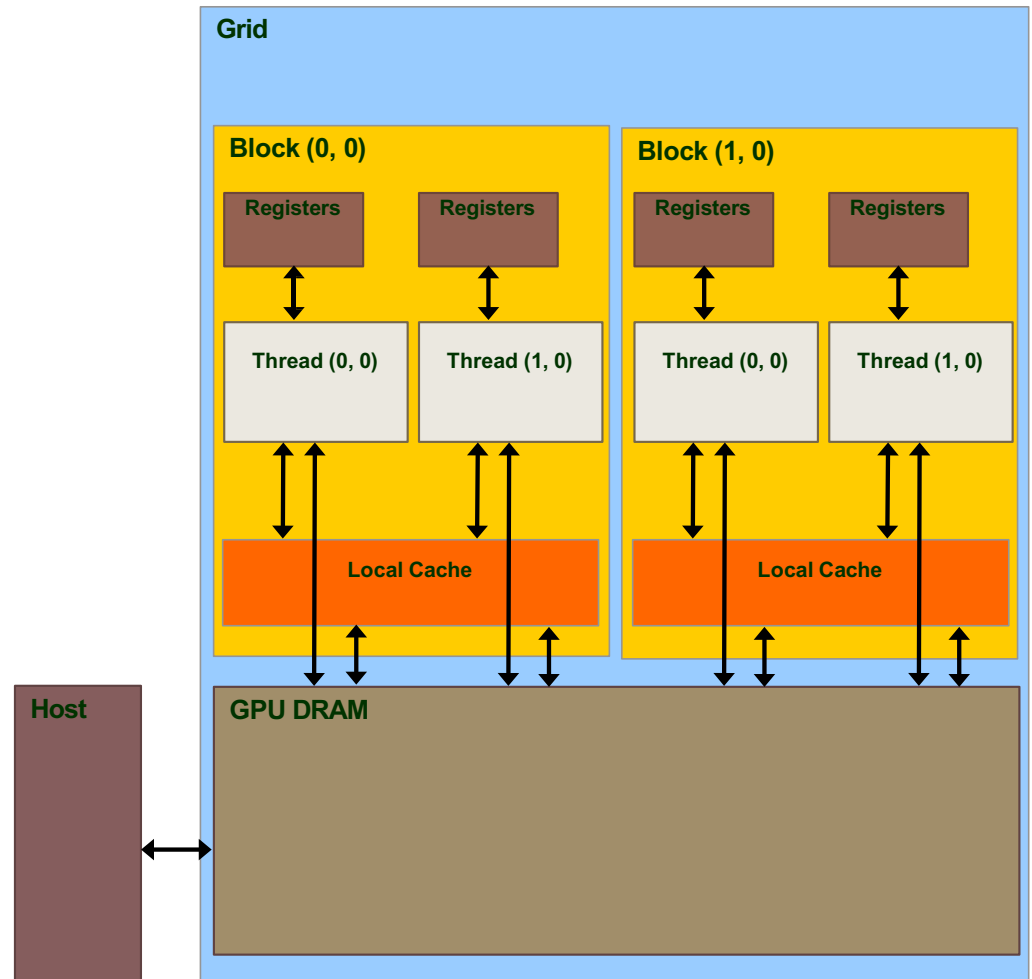
Memory Hierarchy

❑ Hardware perspective

- ❑ Registers
- ❑ Cache
- ❑ Device Memory (aka. GPU DRAM)
- ❑ Host Memory (on CPU)

❑ Software perspective

- ❑ Local Memory
- ❑ Shared Memory
- ❑ Global Memory
- ❑ Constant Memory
- ❑ Texture Memory
- ❑ Unified Memory



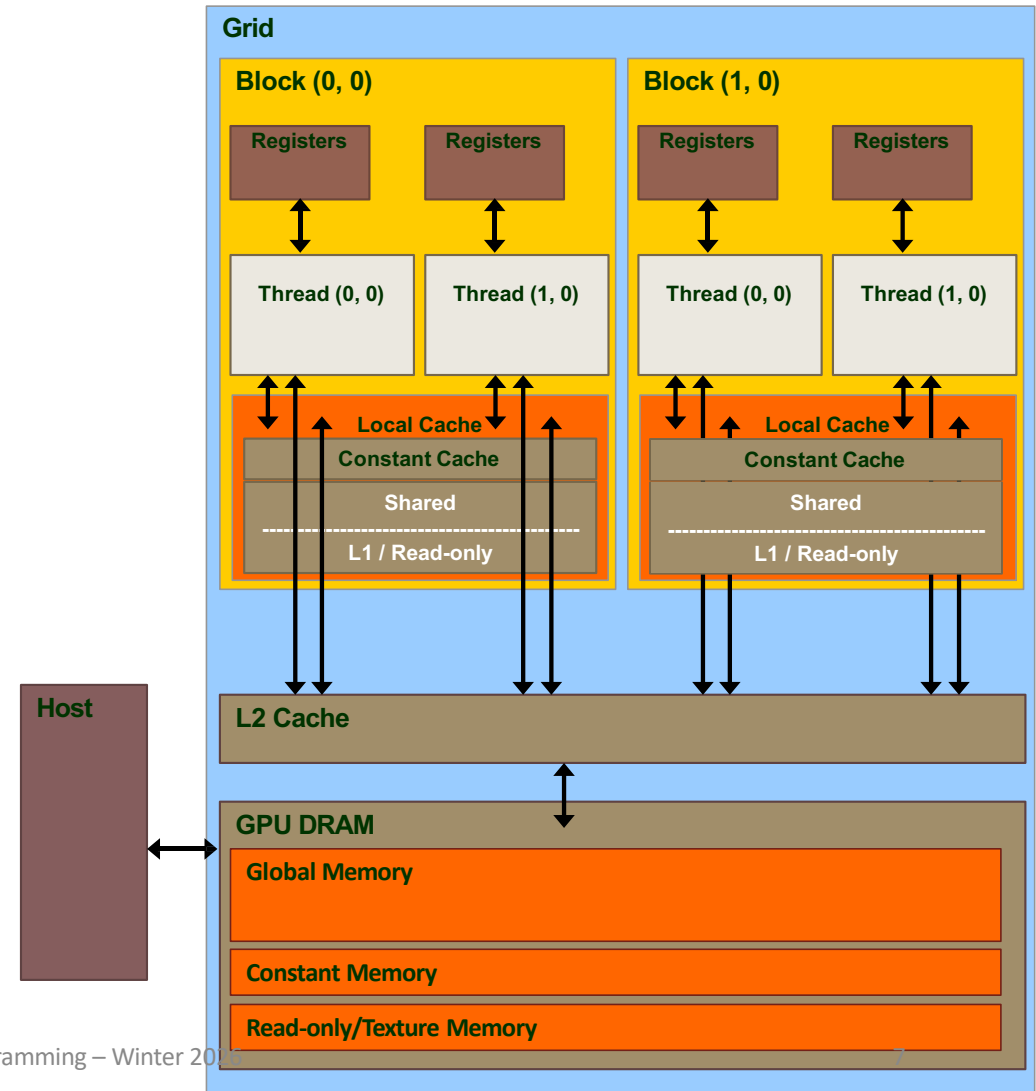
Memory Latencies

- ❑ What is the cost of accessing each area of memory?
- ❑ On chip memory has MUCH lower latency

	Cost (cycles)
Register	1
L1 cache	20-30
L2 cache	~200
Device memory/DRAM	400-800
Host memory (access from GPU)	2000-5000

Volta

- ❑ Each Thread has access to
 - ❑ Registers
 - ❑ Local memory
 - ❑ Main DRAM Memory via L2 cache
 - ❑ Global Memory
 - ❑ Can be read via Unified Data Cache
 - ❑ Constant Memory
 - ❑ Via L2 cache and per block Constant cache
 - ❑ **Unified Shared/L1/Texture Cache**
 - ❑ Same cache used for read only or texture reads
 - ❑ Amount of shared memory configurable at runtime



Cache and Memory Sizes

	Pascal (P100) GP100	Volta (V100) GV100
Register File Size	256KB per SM	256KB per SM
Shared Memory	64KB Dedicated	Configurable up to 96KB
Constant Memory	64KB DRAM 8KB Cache per SM	64KB DRAM 8KB Cache per SM
L1/Read Only Memory	24KB per SM Dedciated	Configurable up to 128KB per SM
L2 Cache Size	4096KB	6144KB
Device Memory	16GB	16GB
DRAM Interface	4096-bit HBM2	4096-bit HBM2

<https://devblogs.nvidia.com/inside-volta/>

Memory Hierarchy

☐ Hardware perspective

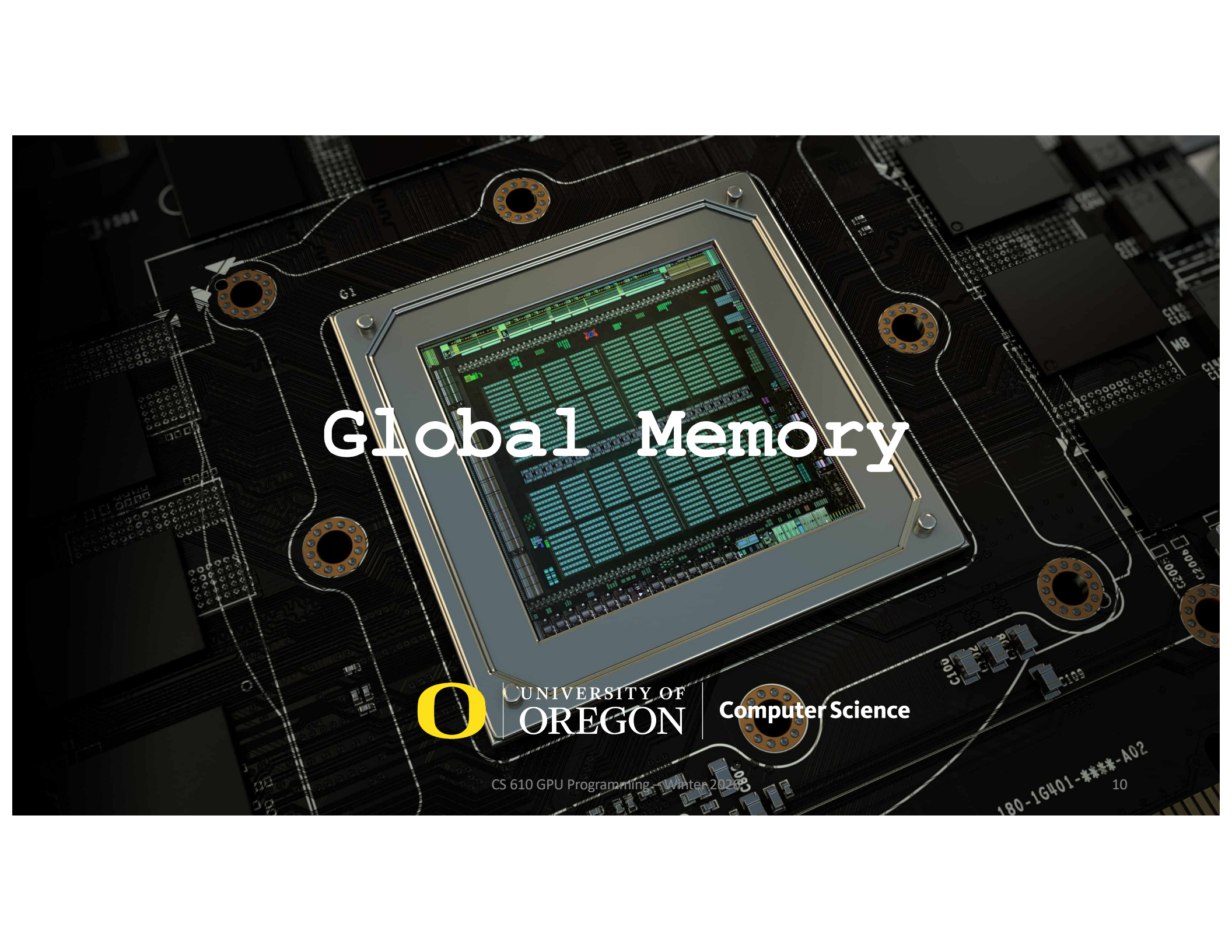
- ☐ Registers
- ☐ Cache
- ☐ Device Memory (aka. GPU DRAM)

☐ Software perspective

- ☐ Local Memory
- ☐ Shared Memory
- ☐ Global Memory
- ☐ Constant Memory
- ☐ Texture Memory
- ☐ Unified Memory

How is software-defined memory mapped to hardware?

How to choose different memory types?



Global Memory



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

10

(Dynamic) Global Memory

- ❑ All memory allocated by `cudaMalloc()`
- ❑ Hardware mapping
 - ❑ Reside in device memory (DRAM)
 - ❑ Cached in L2 (read/write) and L1 (read-only)
- ❑ Lifespan: developer controlled
- ❑ Scope: All threads in all kernels

```
float *a;  
cudaMalloc(&a, N*sizeof(float));  
...  
cudaFree(a);
```

Memory Copying

- ❑ Once memory has been allocated we need to copy data to it and from it.
- ❑ **cudaMemcpy()** transfers memory from the host to device to host and vice versa

```
cudaMemcpy(array_device, array_host,  
            N*sizeof(float), cudaMemcpyHostToDevice);
```

```
cudaMemcpy(array_host, array_device,  
            N*sizeof(float), cudaMemcpyDeviceToHost);
```

- ❑ First argument is always the **destination** of transfer
- ❑ Transfers are relatively slow and should be minimized where possible



```

#define N 2048
#define THREADS_PER_BLOCK 128

__global__ void vectorAdd(float *a, float *b, float *c) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    c[i] = a[i] + b[i];
}

int main(void) {
    float *a, *b, *c;           // host copies of a, b, c
    float *d_a, *d_b, *d_c;    // device copies of a, b, c
    int size = N * sizeof(float);

    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    a = (float *)malloc(size); random_floats(a, N);
    b = (float *)malloc(size); random_floats(b, N);
    c = (float *)malloc(size);

    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    vectorAdd <<<N / THREADS_PER_BLOCK, THREADS_PER_BLOCK >>>(d_a, d_b, d_c);

    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}

```

Define macros

Define kernel

Define pointer variables

Allocate GPU memory

Allocate host memory and
initialise contents

Copy input data to the
device

Launch the kernel

Copy data back to host

Clean up

Static Global Memory

- ❑ How do we declare a large array on the host without using malloc?
 - ❑ Statically allocate using compile time size

```
int array[N];
```

- ❑ We can do the same on the device. i.e.
 - ❑ Just like when applied to a function
 - ❑ Only available on the device
 - ❑ Must use `cudaMemcpyToSymbol()`
 - ❑ Must be a global variable

```
__device__ int array[N];
```



```

#define N 2048
#define THREADS_PER_BLOCK 128

__device__ float d_a[N];
__device__ float d_b[N];
__device__ float d_c[N];

__global__ void vectorAdd() {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    d_c[i] = d_a[i] + d_b[i];
}

int main(void) {
    float *a, *b, *c;           // host copies of a, b, c
    int size = N * sizeof(float);

    a = (float *)malloc(size); random_floats(a, N);
    b = (float *)malloc(size); random_floats(b, N);
    c = (float *)malloc(size);

    cudaMemcpyToSymbol(d_a, a, size);
    cudaMemcpyToSymbol(d_b, b, size);

    vectorAdd <<<N / THREADS_PER_BLOCK, THREADS_PER_BLOCK >>>();

    cudaMemcpyFromSymbol(c, d_c, size);

    free(a); free(b); free(c);
    return 0;
}

```

Define macros

Statically allocate GPU memory

Define kernel

Define pointer variables

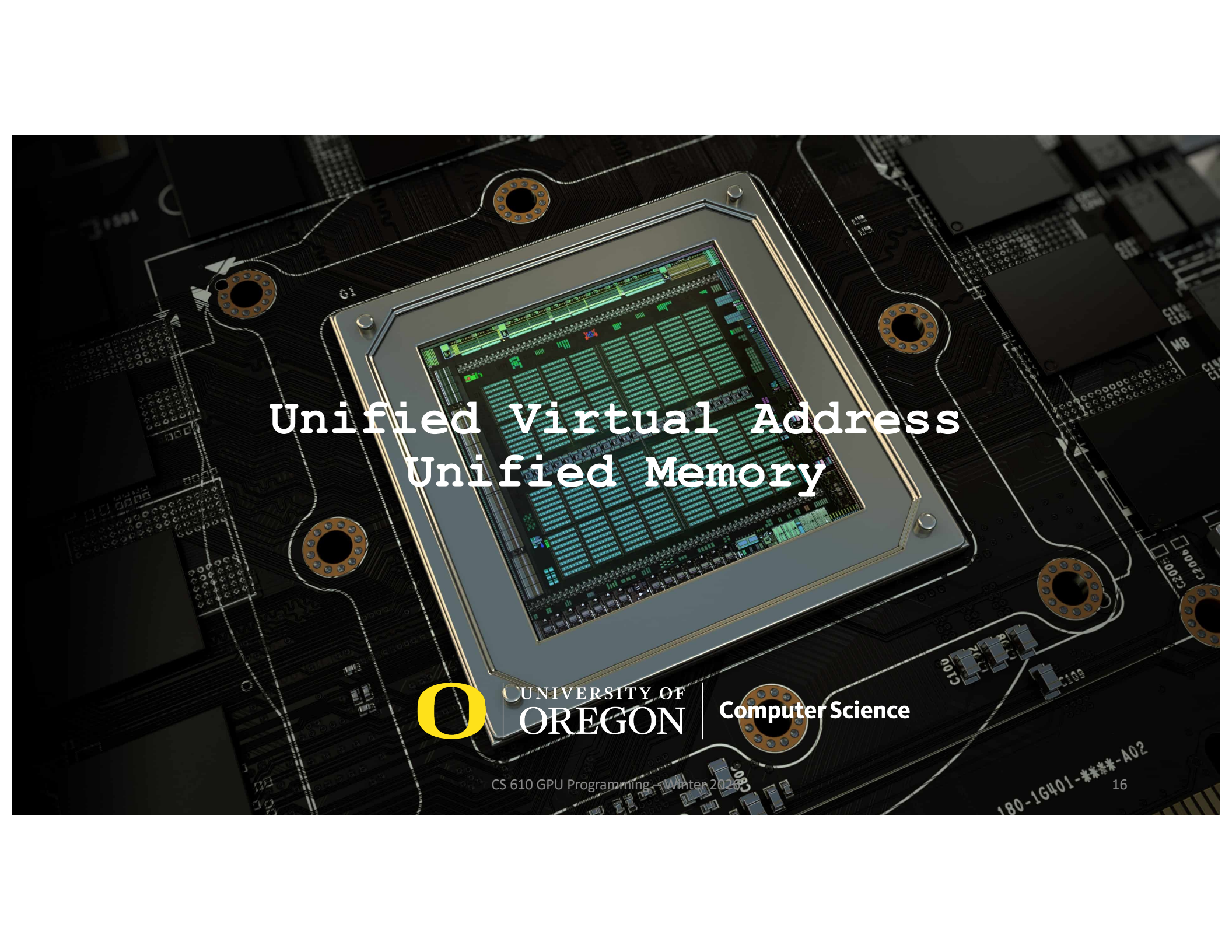
Allocate host memory and initialise contents

Copy input data to the device

Launch the kernel

Copy data back to host

Clean up



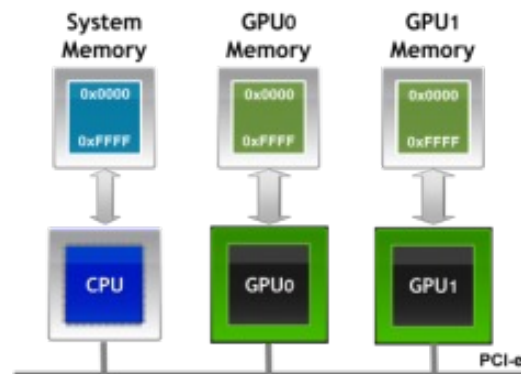
Unified Virtual Address Unified Memory



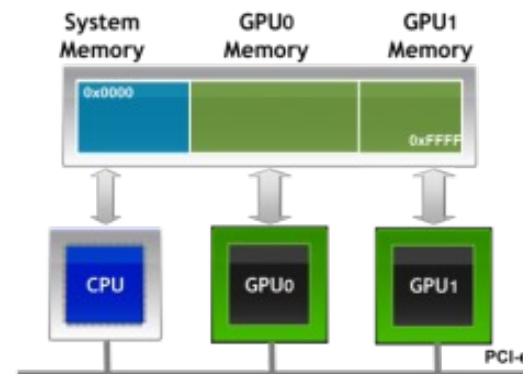
Unified Virtual Address (UVA)

- A single memory address space is used for the host and all the devices
- Enabled by default for 64-bit program
- Addresses in UVA belong to the same space, so they can be managed together
- If you use a modern system, all GPU/CPU memory allocations are in UVA by default

No UVA: Multiple Memory Spaces



UVA: Single Address Space



Exercise

- Given address of a CPU and a GPU memory buffer, which of the following is possible?

Case 1

UVA Enabled

CPU buffer address: 0x236790FF

GPU buffer address: 0x236790FF

Case 2

UVA Enabled

CPU buffer address: 0x236790FF

GPU buffer address: 0x745790AD

Case 3

UVA not available

CPU buffer address: 0x236790FF

GPU buffer address: 0x236790FF

Case 4

UVA not available

CPU buffer address: 0x236790FF

GPU buffer address: 0x745790AD



Unified Virtual Address Space

cudaPointerGetAttributes ([cudaPointerAttributes](#)* attributes, const void* ptr)

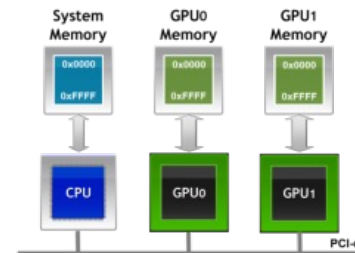
```
int * dev_ptr;
cudaMalloc((void**)&dev_ptr, 100);
int * host_ptr = (int*)malloc(100);

cudaPointerAttributes attr1;
cudaPointerGetAttributes(attr1, dev_ptr);
// attr.cudaMemoryType == cudaMemoryTypeDevice

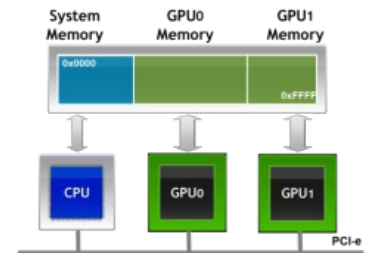
cudaPointerAttributes attr2;
cudaPointerGetAttributes(attr2, host_ptr);
// attr.cudaMemoryType == cudaMemoryTypeHost
```

```
struct cudaPointerAttributes {
    enum cudaMemoryType
        type;
    int device;
    void *devicePointer;
    void *hostPointer;
}
```

No UVA: Multiple Memory Spaces



UVA: Single Address Space

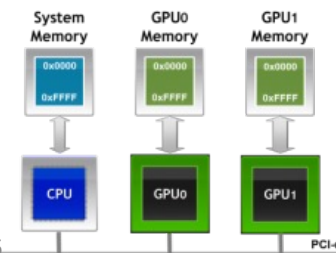


Unified Virtual Address Space

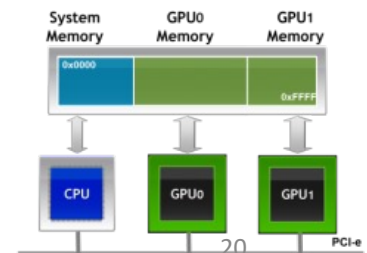
- Let CUDA figure out the copy direction between CPU and GPU based on pointer type
- Pass `cudaMemcpyDefault` to `cudaMemcpy`

```
int * dev_ptr;  
cudaMalloc((void**)&dev_ptr, 100);  
int * host_ptr = (int*)malloc(100);  
  
// following two are equivalent  
cudaMemcpy(dev_ptr, host_ptr, 100, cudaMemcpyHostToDevice)  
cudaMemcpy(dev_ptr, host_ptr, 100, cudaMemcpyDefault)  
  
// following two are equivalent  
cudaMemcpy(host_ptr, dev_ptr, 100, cudaMemcpyDeviceToHost)  
cudaMemcpy(host_ptr, dev_ptr, 100, cudaMemcpyDefault)
```

No UVA: Multiple Memory Spaces

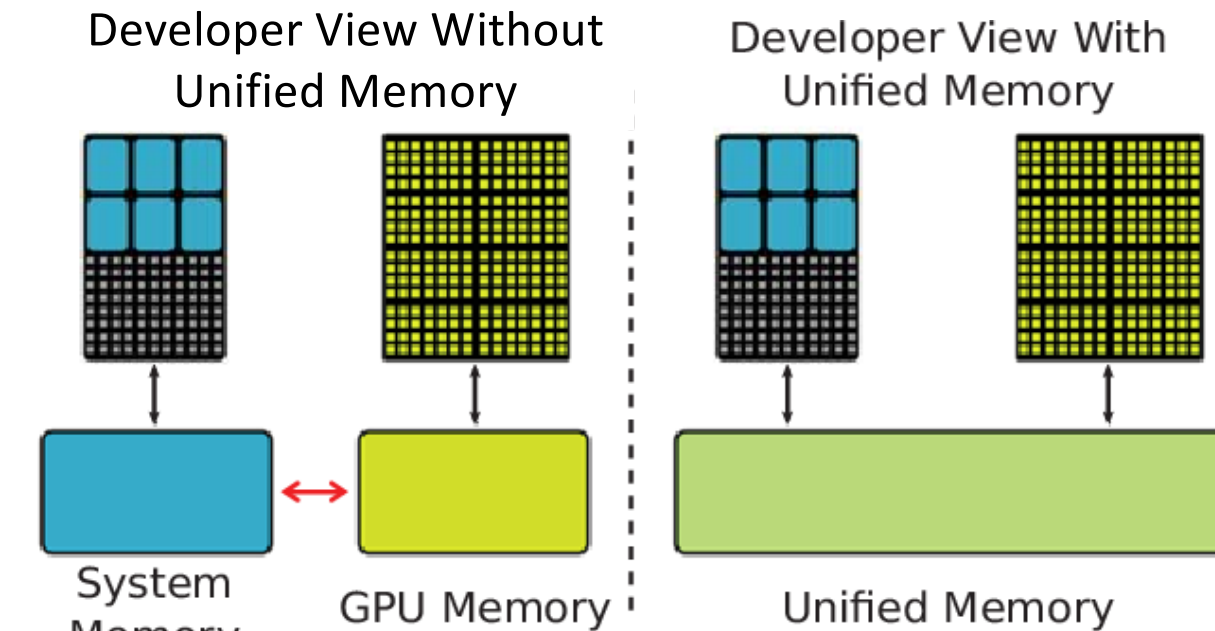


UVA: Single Address Space



Unified Memory

- Unified Memory = CUDA Managed Memory
- Accessible by both CPU and GPU



Unified Memory

- Unified Memory = CUDA Managed Memory
- Accessible by both CPU and GPU

```
__global__ void AplusB(int *ret, int a, int b) {  
    ret[threadIdx.x] = a + b + threadIdx.x;  
}  
  
int main() {  
    int *ret;  
    cudaMalloc(&ret, 1000 * sizeof(int));  
    AplusB<<< 1, 1000 >>>(ret, 10, 100);  
    int *host_ret = (int *)malloc(1000 * sizeof(int));  
    cudaMemcpy(host_ret, ret, 1000 * sizeof(int),  
    cudaMemcpyDefault);  
    for(int i = 0; i < 1000; i++)  
        printf("%d: A+B = %d\n", i, host_ret[i]);  
    free(host_ret);  
    cudaFree(ret);  
}
```

```
__global__ void AplusB(int *ret, int a, int b) {  
    ret[threadIdx.x] = a + b + threadIdx.x;  
}  
  
int main() {  
    int *ret;  
    cudaMallocManaged(&ret, 1000 * sizeof(int));  
    AplusB<<< 1, 1000 >>>(ret, 10, 100);  
    for(int i = 0; i < 1000; i++)  
        printf("%d: A+B = %d\n", i, ret[i]);  
    cudaFree(ret);  
}
```

No explicit memory copy calls

Unified Memory Example

C Code

```
void sortfile(FILE *fp, int N) {  
    char *data;  
    data = (char *)malloc(N);  
  
    fread(data, 1, N, fp);  
  
    qsort(data, N, 1, compare);  
  
    use_data(data);  
  
    free(data);  
}
```

CUDA (6.0+) Code

```
void sortfile(FILE *fp, int N) {  
    char *data;  
    cudaMallocManaged(&data, N);  
  
    fread(data, 1, N, fp);  
  
    gpu_qsort(data, N, 1, compare);  
    cudaDeviceSynchronize();  
  
    use_data(data);  
  
    cudaFree(data);  
}
```



Unified Memory

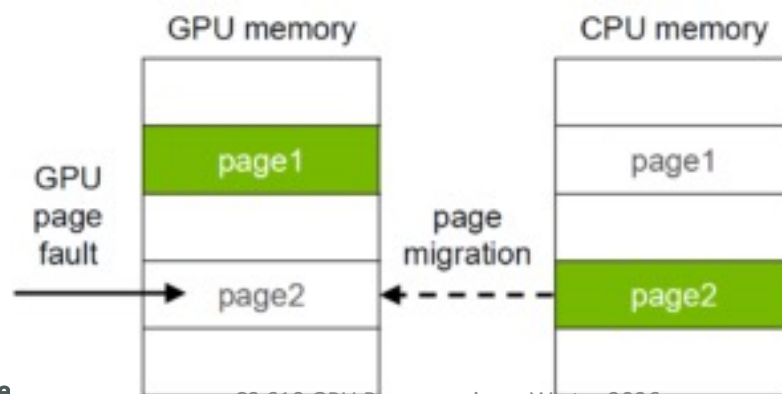
- hint

Unified Virtual Address vs. Unified Memory

- UVA
 - Enables unified address space
 - Developers need to manage CPU/GPU memory allocation separately
 - Need to manage two copies of a data (need copy to sync data)
- UM
 - UM is only possible when UVA is enabled (UM depends on UVA)
 - Developers only need to manage one memory allocation
 - “single-pointer-to-data” model (copy is done internally by CUDA)

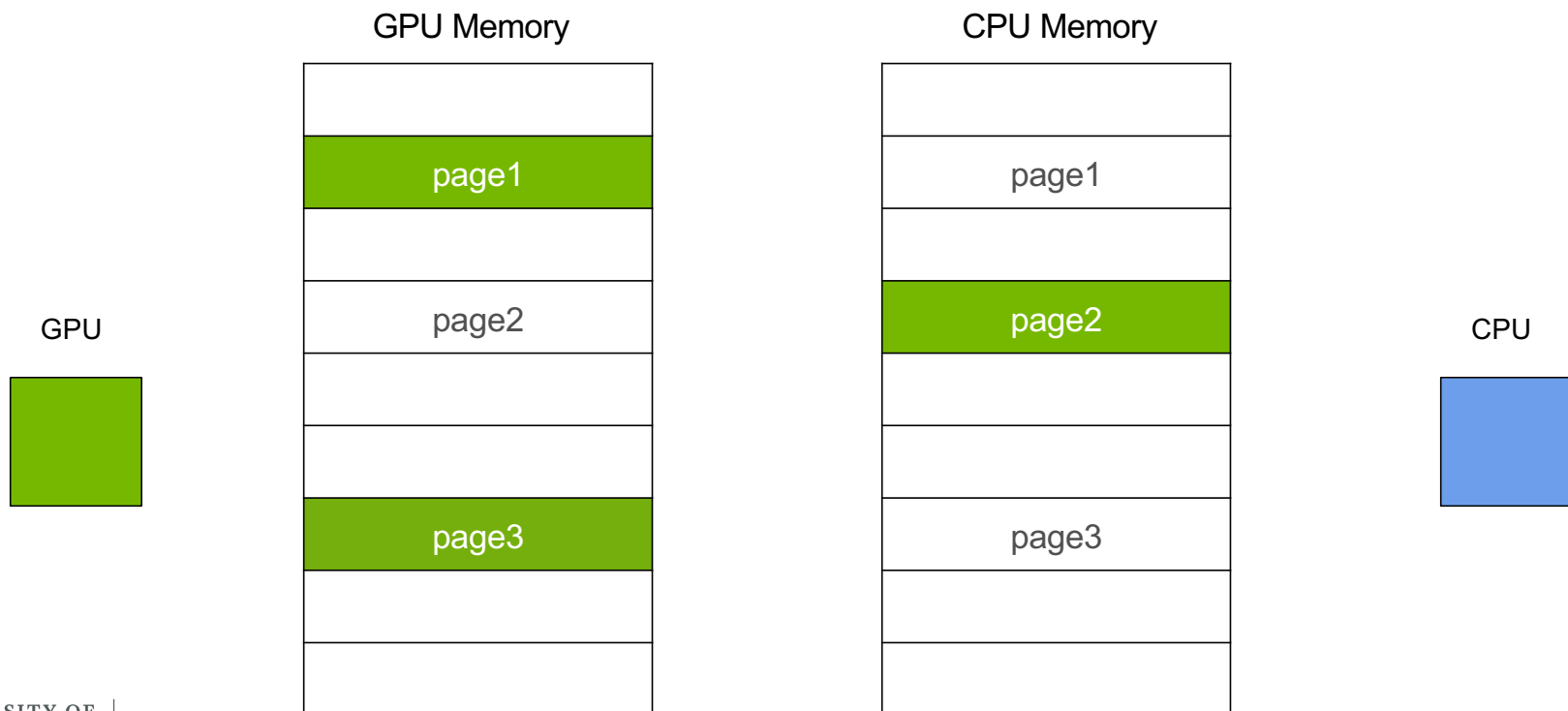
How is unified memory designed?

- No change to the hardware: still have two separate memory spaces for CPU and GPU
- CUDA internally handles memory copies
- Fine-grain memory copies are used (**page fault**), so both CPU and GPU can access the data at the same time



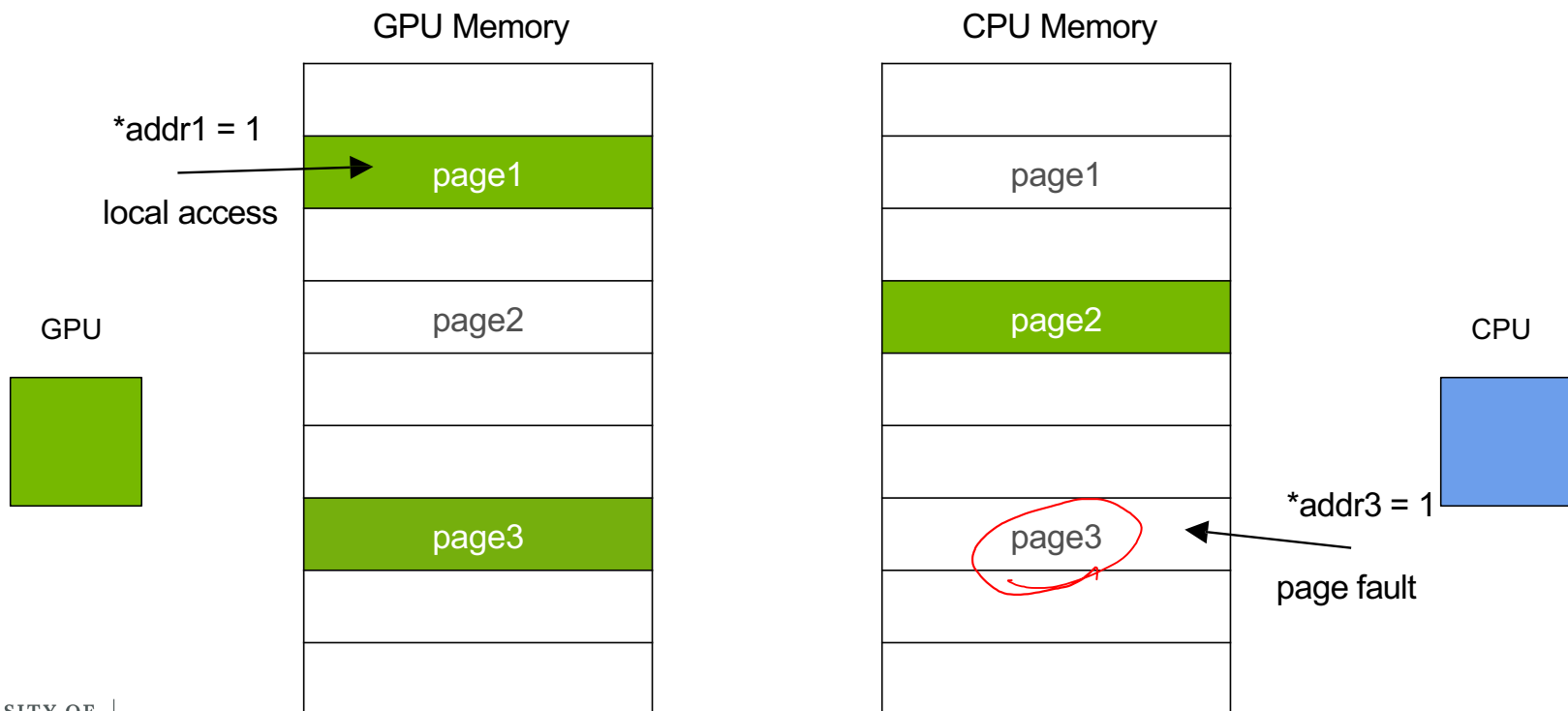
UNIFIED MEMORY FUNDAMENTALS

On-Demand Migration



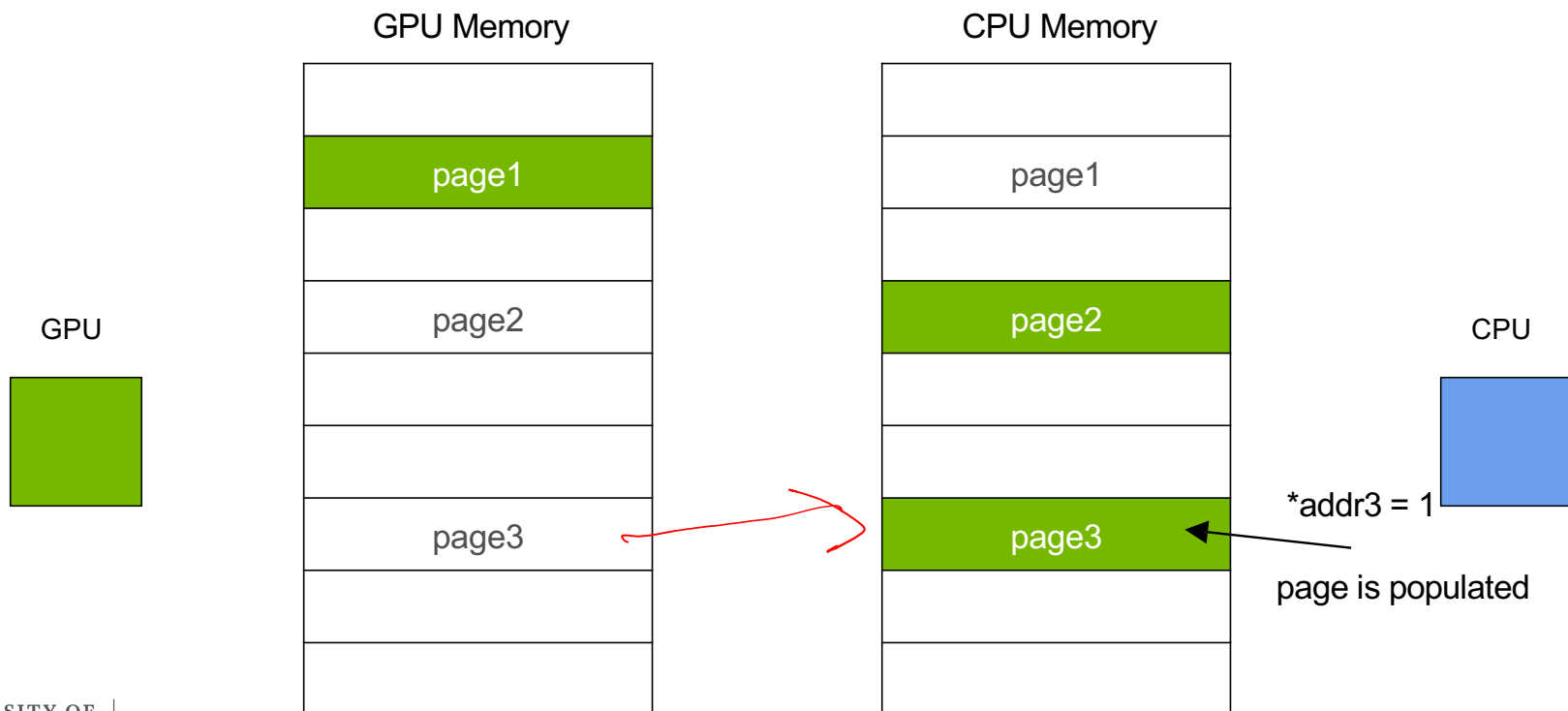
UNIFIED MEMORY FUNDAMENTALS

On-Demand Migration



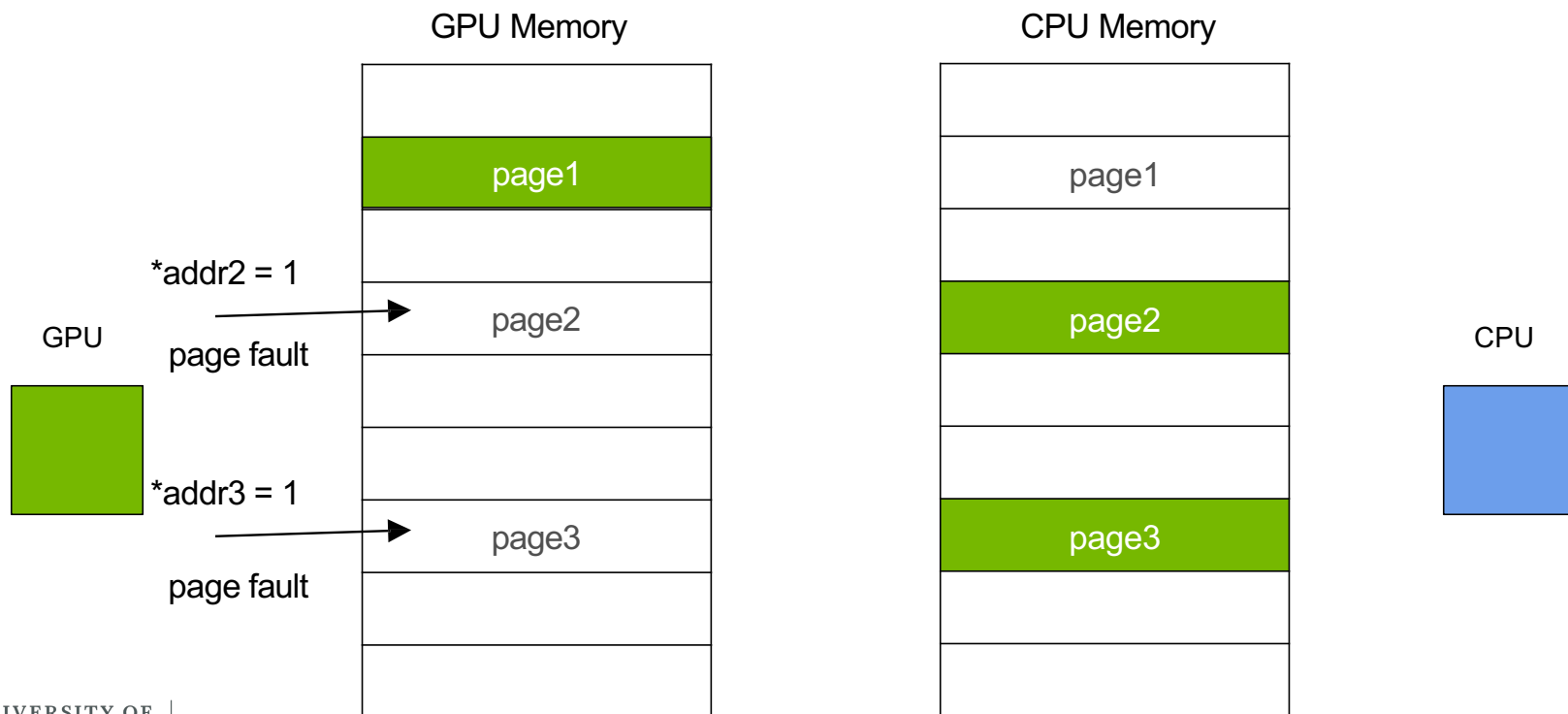
UNIFIED MEMORY FUNDAMENTALS

On-Demand Migration



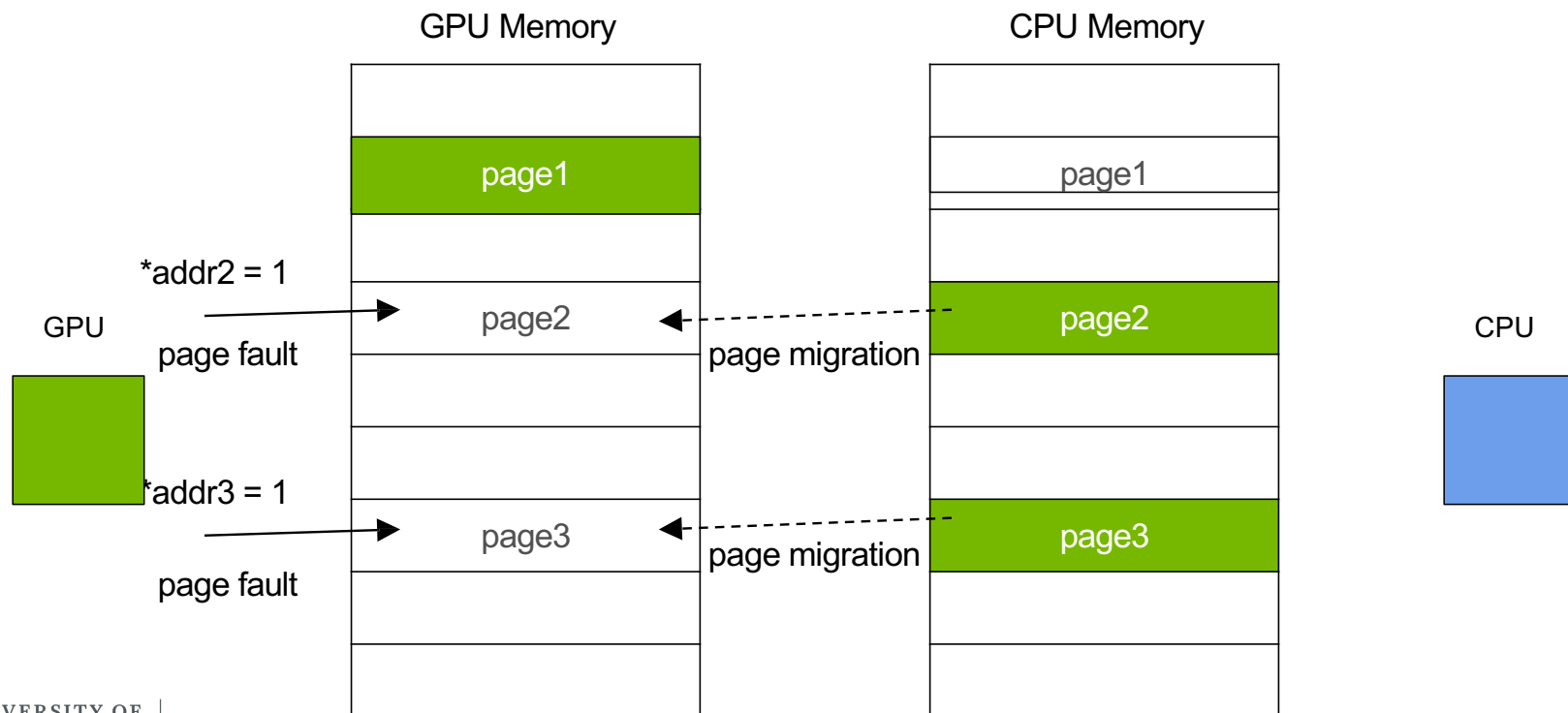
UNIFIED MEMORY FUNDAMENTALS

On-Demand Migration



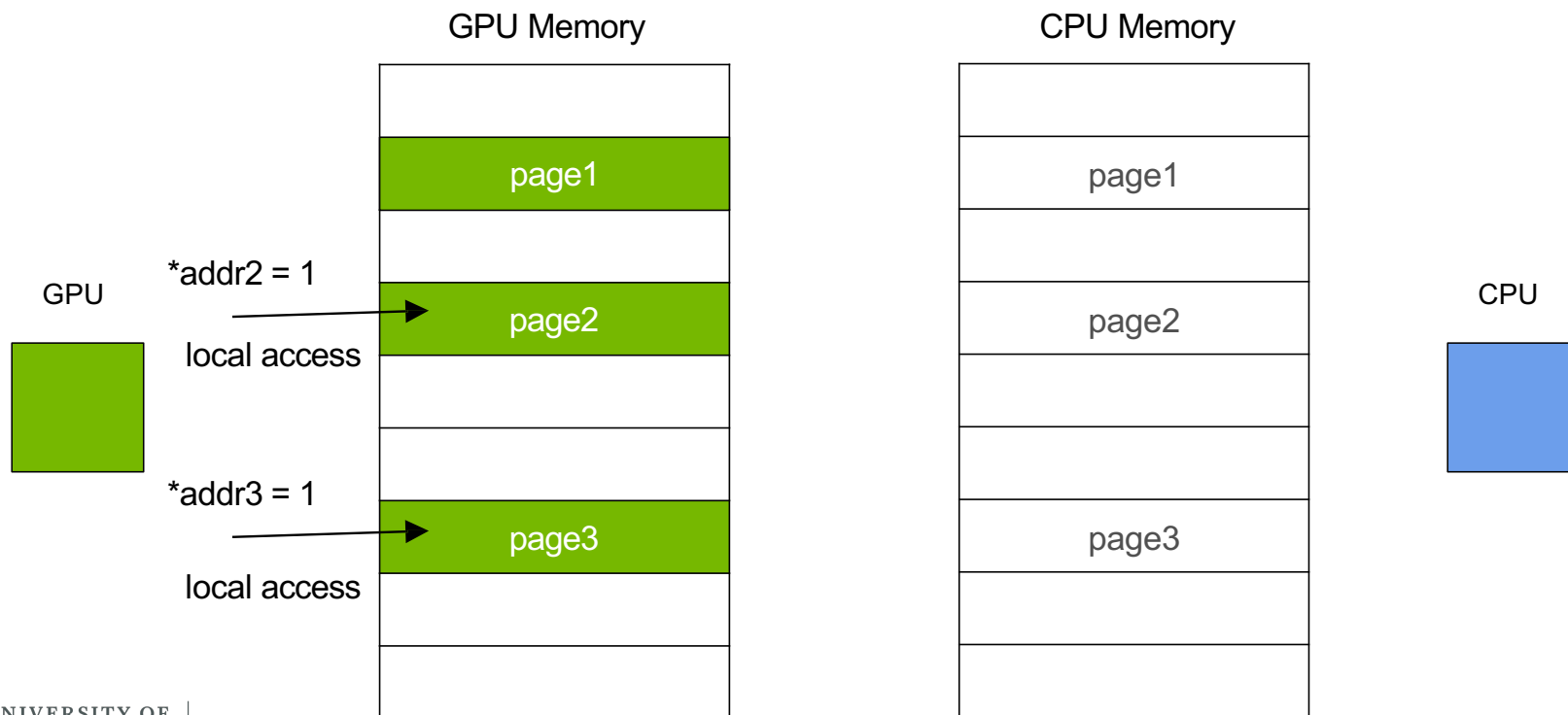
UNIFIED MEMORY FUNDAMENTALS

On-Demand Migration



UNIFIED MEMORY FUNDAMENTALS

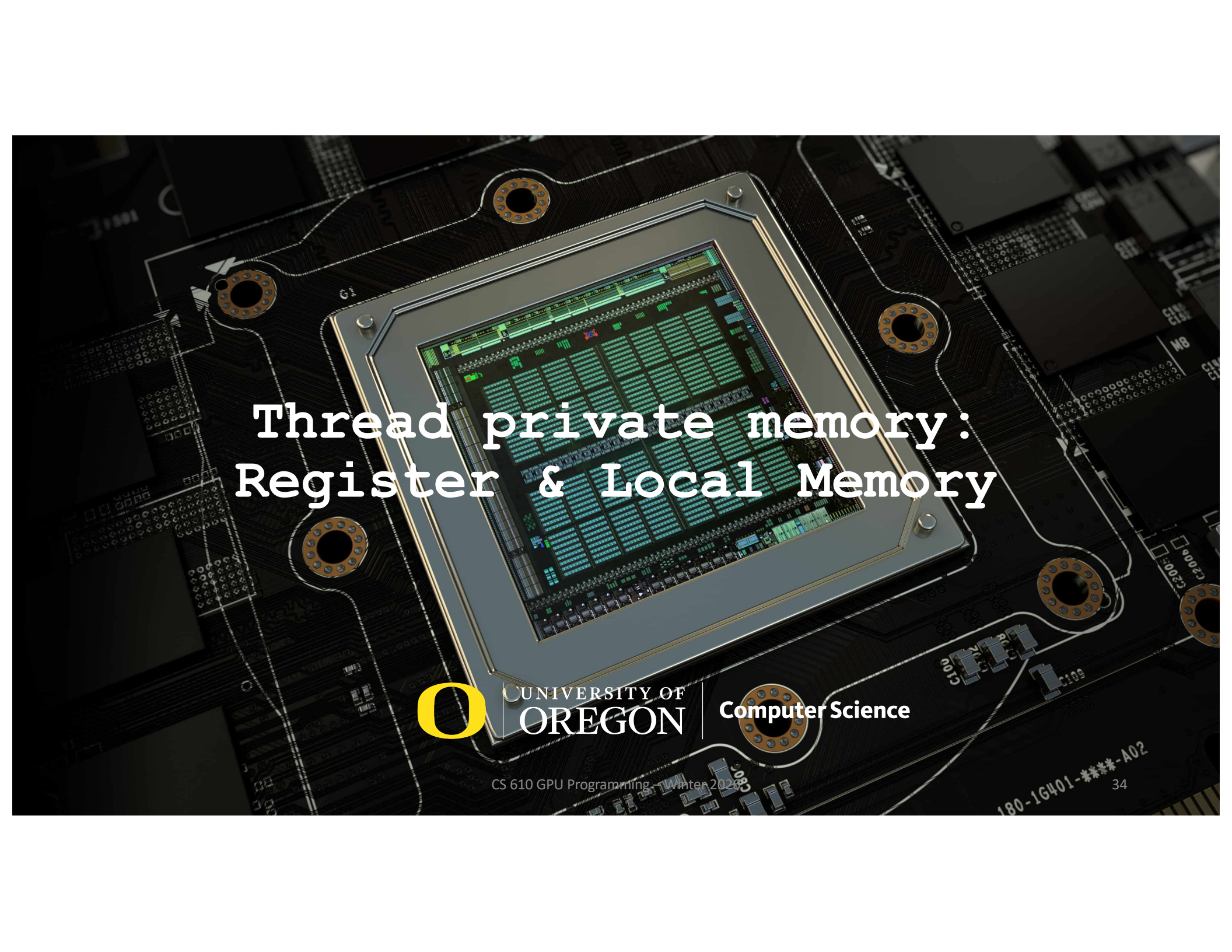
On-Demand Migration



Implications of CUDA Unified (Managed) Memory

- ❑ Simpler porting of code
- ❑ Memory is only *virtually* unified
 - ❑ GPU still has discrete memory
 - ❑ It still has to be transferred via PCIe (or NVLINK)
- ❑ Easier management of data to and from the device
 - ❑ Explicit memory movement is not required
 - ❑ Similar to the way the OS handles virtual memory
- ❑ Issues
 - ❑ Requires look ahead and paging to ensure memory is in the correct place (and synchronized)
 - ❑ It is not as fast as hand tuned code which has finer explicit control over transfers
- ❑ *We will manage memory movement ourselves!*





Thread private memory: Register & Local Memory



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming: Winter 2026

34

Thread private memory

- ❑ Each thread has **private** memory
- ❑ **Not** accessible by other threads
- ❑ All variables defined in
 - ❑ `__global__` functions
 - ❑ `__device__` functions
- ❑ Each thread has their **own** copy
- ❑ Stored in **registers** and **local memory**

```
__global__ void localMemoryExample (int * input)
{
    int a, b, c;
    int n = 100;
    int index;

    int myArray1[4];
    int myArray2[4];
    int myArray3[100];

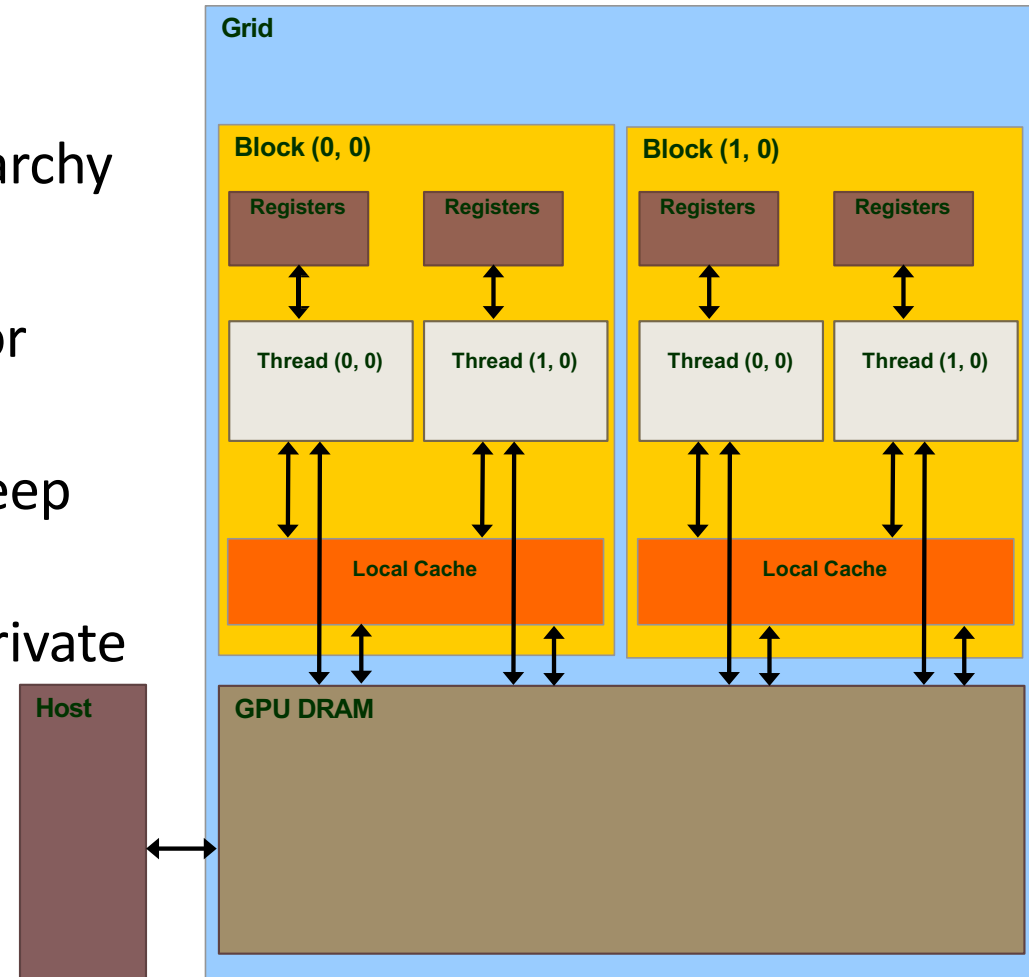
    int* dym_arr = (int*)malloc(n*sizeof(int));

}
```

<https://stackoverflow.com/questions/10297067/in-a-cuda-kernel-how-do-i-store-an-array-in-local-thread-memory>

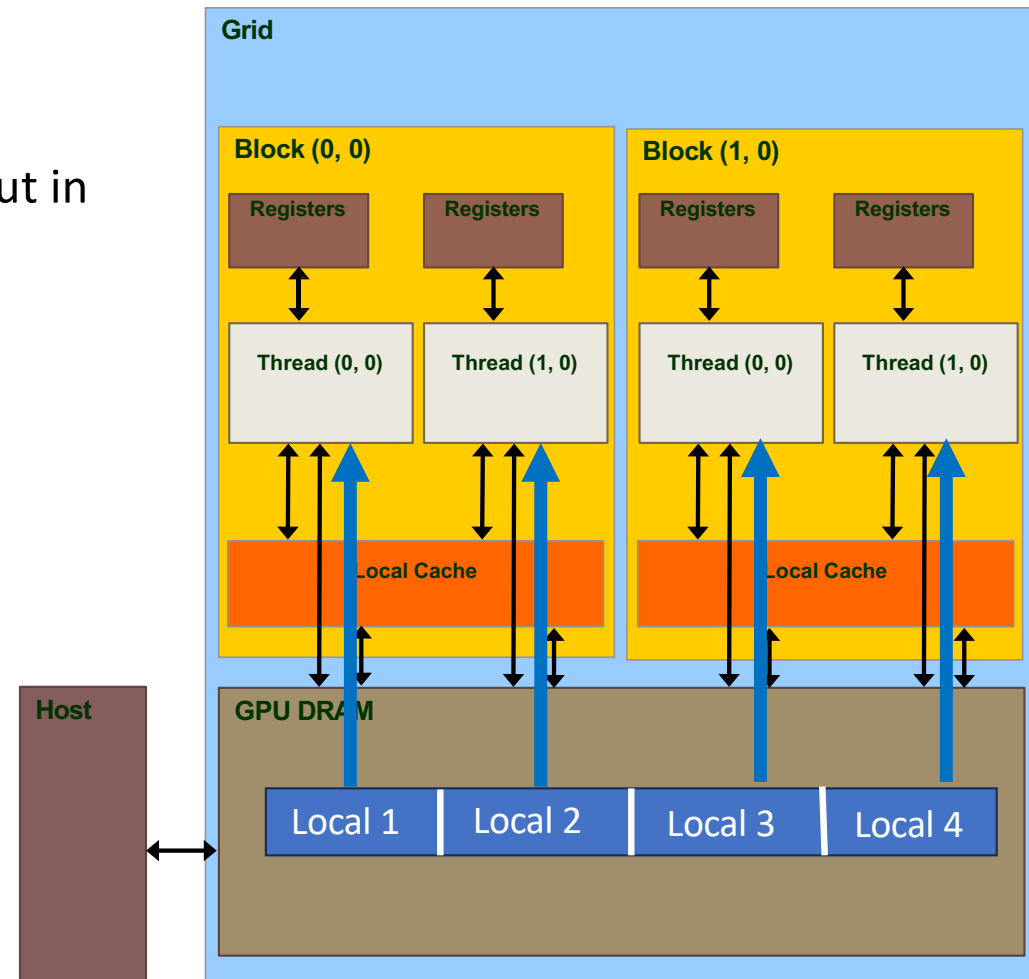
Registers

- **Fastest** memory in GPU memory hierarchy
- **1 cycle access latency**
- All operands need to be in registers for calculation
- Computation is **more efficient** if we keep more data in registers
- If possible, compiler will put thread private data in registers
- Registers are **very limited** per thread



Local Memory

- If some thread private memory cannot be put in registers, they are put in **local memory**
- Local memory does **not physically exist**
- Mapped to reserved area in **global memory**
- Global (local) memory is much slower than registers
- 400-800 cycles access latency



Register spilling

- If some thread private memory cannot be put in registers, it is called **register spilling**
 - General Rules
 - Scalar variable: registers (if under limit)
 - Fix-size array
 - Use constant index: registers (if under limit)
 - $A[10] = A[20] + A[30]$
 - Use non-const index: local memory
 - $A[i] = A[j] + A[k]$
 - Dynamic-size array: local memory
- Registers are not addressable**

```
__global__ void localMemoryExample (int * input)
{
    int a, b, c; ←Register
    int n = 100; ←Register
    int index; ←Register

    int myArray1[4]; ←Register
    int myArray2[4]; ←Local Memory
    int myArray3[100]; ←Register (could have partial spill)

    int* dym_arr = (int*)malloc(n*sizeof(int));
                                ←Local Memory
    index = input[threadIdx.x];
    a = myArray1[0];
    b = myArray2[index];
    c = myArray3[90];

    dym_arr[10] = a;
}
```

Register spilling

- Compiler makes the final decision
- Extra register spilling hurts performance

<https://stackoverflow.com/questions/10297067/in-a-cuda-kernel-how-do-i-store-an-array-in-local-thread-memory>

```
__global__ void localMemoryExample (int * input)
{
    int a, b, c; ←Register
    int n = 100; ←Register
    int index; ←Register

    int myArray1[4]; ←Register
    int myArray2[4]; ←Local Memory
    int myArray3[100]; ←Register (could have partial spill)

    int* dym_arr = (int*)malloc(n*sizeof(int));
                                ←Local Memory
    index = input[threadIdx.x];
    a = myArray1[0];
    b = myArray2[index];
    c = myArray3[90];

    dym_arr[10] = a;
}
```

Minimizing Register spilling

- Limit use of scalar variables
- If using array is necessary, try to only use constant index
- if non-const index is unavoidable, try to use shared memory

```
__global__ void localMemoryExample (int * input)
{
    int a, b, c; ←Register
    int n = 100; ←Register
    int index; ←Register

    int myArray1[4]; ←Register
    int myArray2[4]; ←Local Memory
    int myArray3[100]; ←Register (could have partial spill)

    int* dym_arr = (int*)malloc(n*sizeof(int));
                                ←Local Memory
    index = input[threadIdx.x];
    a = myArray1[0];
    b = myArray2[index];
    c = myArray3[90];

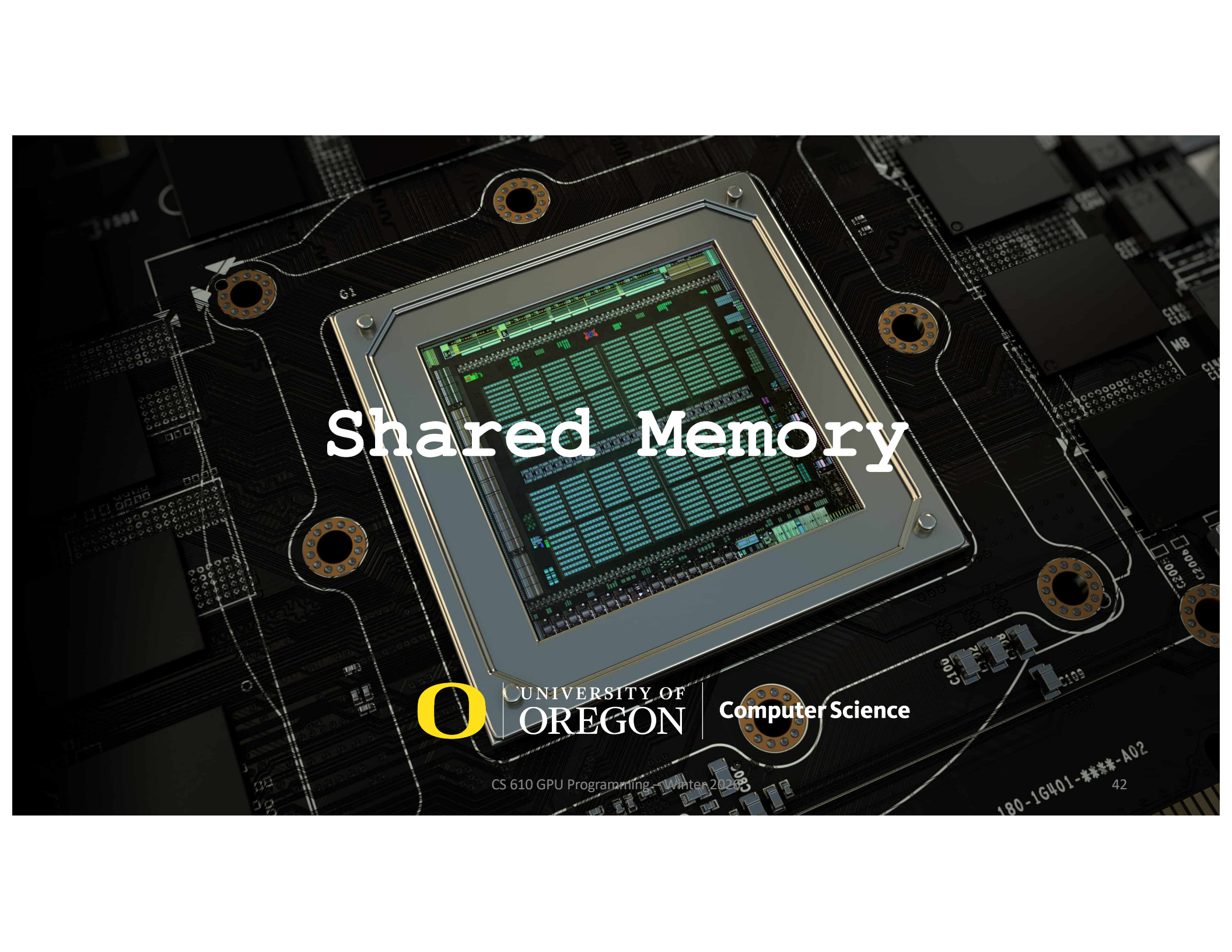
    dym_arr[10] = a;
}
```

Private memory and variables

- All automatic variables declared inside the kernel are in private memory
 - May or may not be on register (why?)
 - Can only be access by device thread
 - Lifespan: kernel execution or device function execution (difference?)
- Not all private memory are for automatic variables
 - One rare case: static variable in device function
 - Lifespan: whole program

```
__device__ int add(int a, int b) {  
    int c = a + b;  
    return c;  
}  
  
__global__ void my_kernel() {  
    int data;  
  
    int *ptr1 = &my_global;  
    int *ptr2 = &my_local;  
}
```

```
__device__ int foo() {  
    static int counter = 0;  
    counter++;  
    return counter;  
}  
  
__global__ void kernel(int *out) {  
    int tid = threadIdx.x;  
    out[tid] = foo();  
}
```



Shared Memory



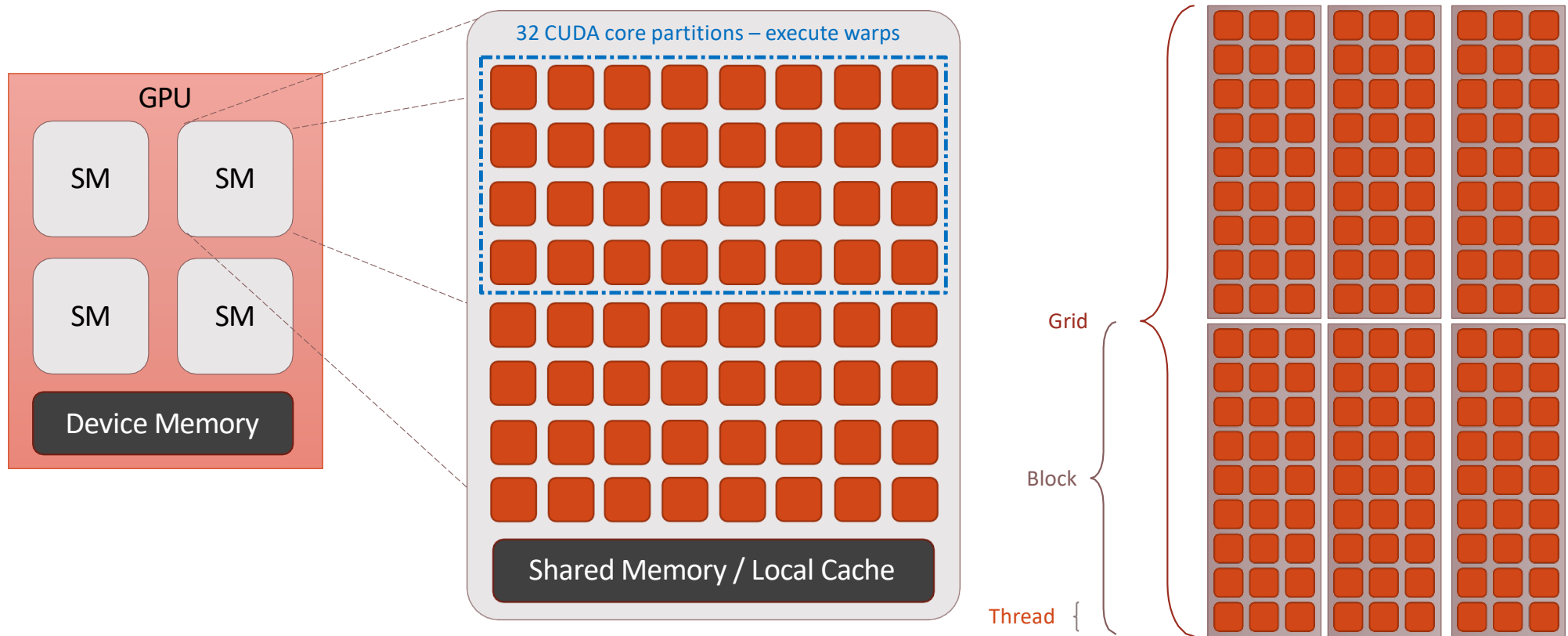
UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

42

Grids, Blocks, Warps & Threads

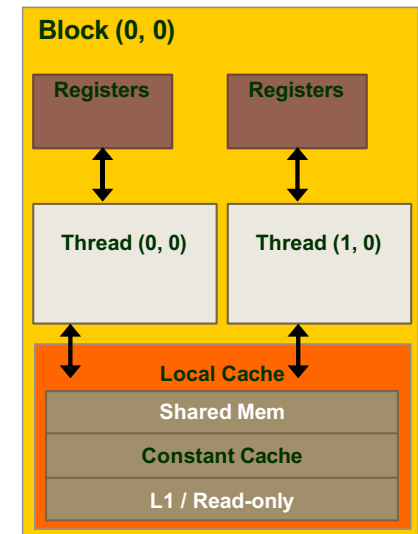


Shared Memory

- ❑ Architecture Details
 - ❑ Shared Memory is dedicated
- ❑ Its just another Cache, right?
 - ❑ User programable
 - ❑ Requires manually loading and synchronising data

A100 SM on-chip storage (192 KB total)

Shared Memory (explicit)	L1 Cache (implicit)
-----------------------------	------------------------



Shared Memory

- ❑ Performance

 - ❑ Shared memory is very fast

 - ❑ Bandwidth > 1 TB/s

- ❑ Block level computation

 - ❑ Allows data to be shared between threads in the same block

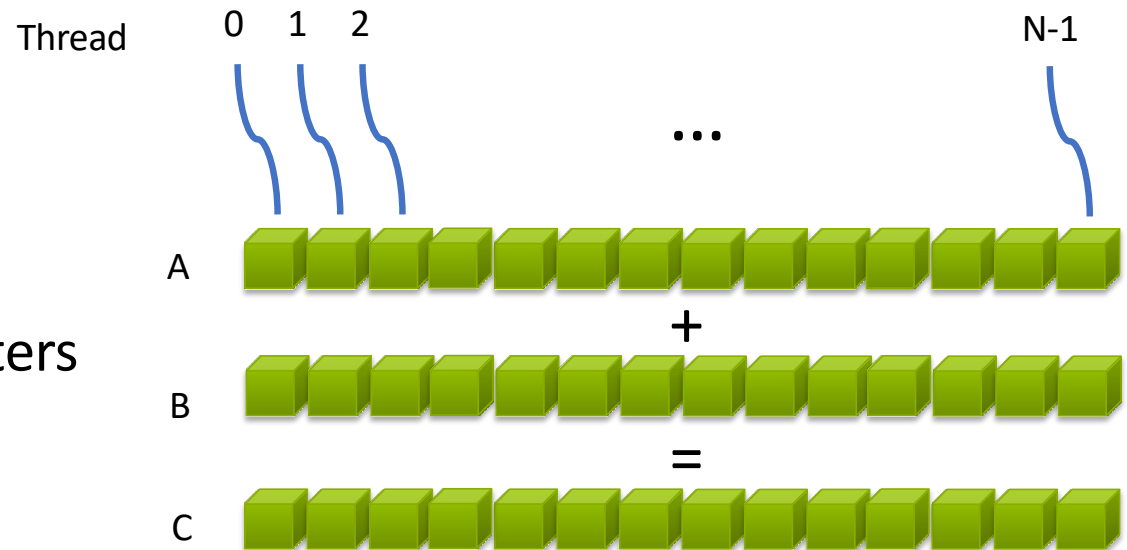
 - ❑ User configurable cache at the thread block level

Vector Add

$$A+B=C$$

Each thread:

1. Load $A[i]$ to register
2. Load $B[i]$ to register
3. Compute $A[i]+B[i]$ in registers
4. Store results to $C[i]$



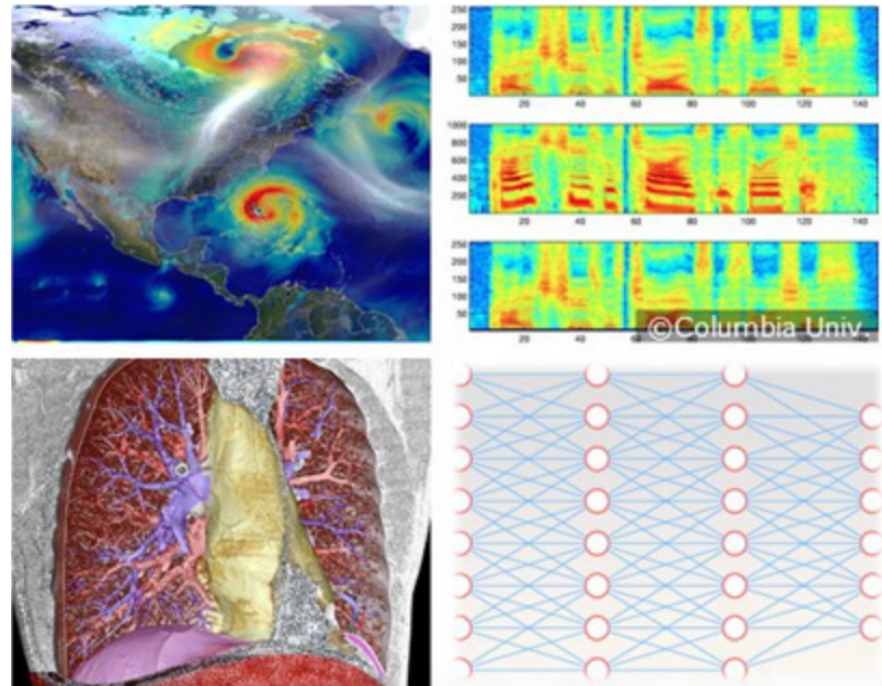
Total amount of data: $2N$ input + N output

Total memory cost: $2N$ loads + N stores



Stencil operations

- Scientific Simulations
 - Fluid Dynamics
 - Thermal Analysis
 - Electromagnetic Field Analysis
 - Climate / Weather
 - Seismic Imaging
 - etc.
- Signal Processing
 - Audio, Sonar
 - Radar, Radio Telescopes
 - etc.
- Image / Volume Data Processing
 - Retouch
 - Data Compression
 - Recognition
 - Medical Diagnosis (Biopsy, CT, MRI, ...)
 - etc.
- Machine Learning
 - Deep Learning (Convolutional Neural Networks



<https://www.nec.com/en/global/solutions/hpc/articles/tech04.html>

Stencil operations

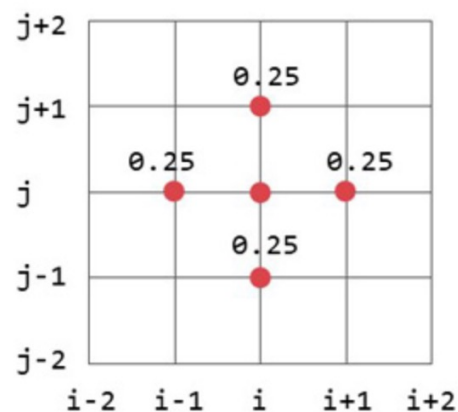
Laplace Equation

$$\frac{\partial}{\partial x^2} A + \frac{\partial}{\partial y^2} A = 0$$

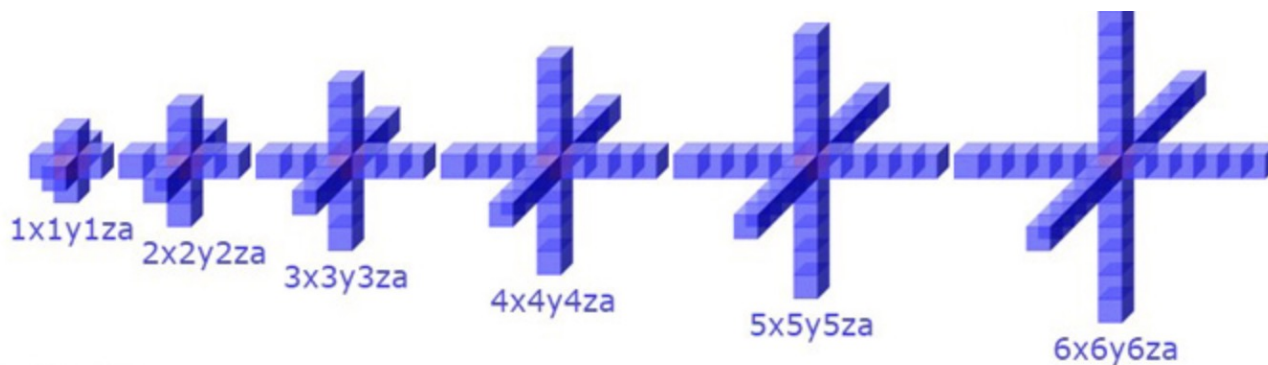


discretization
finite-difference

$$b_{i,j} = \frac{1}{4} (a_{i,j-1} + a_{i-1,j} + a_{i+1,j} + a_{i,j+1})$$



2D Stencil

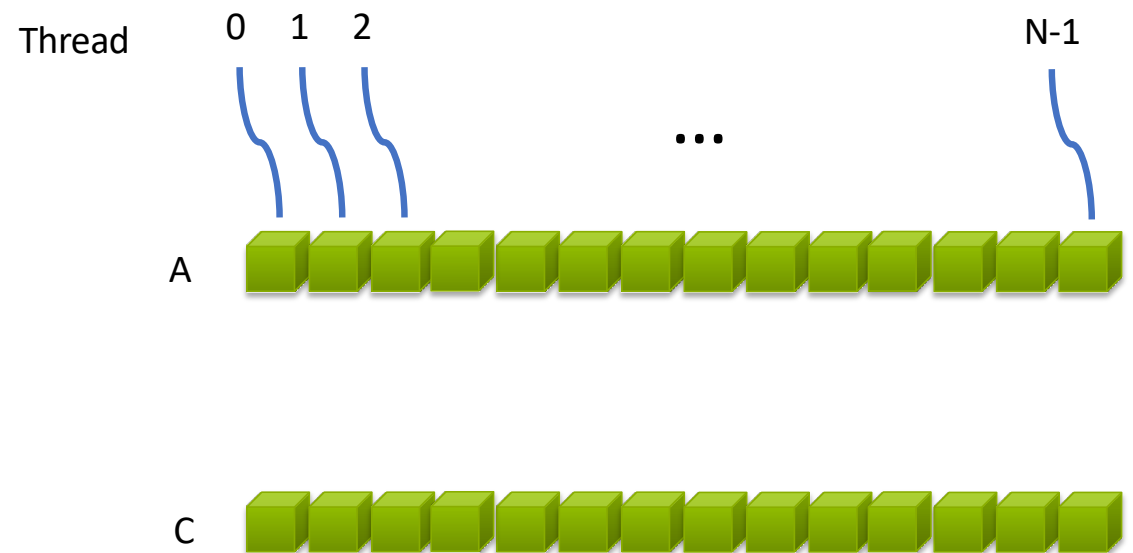


3D Stencil



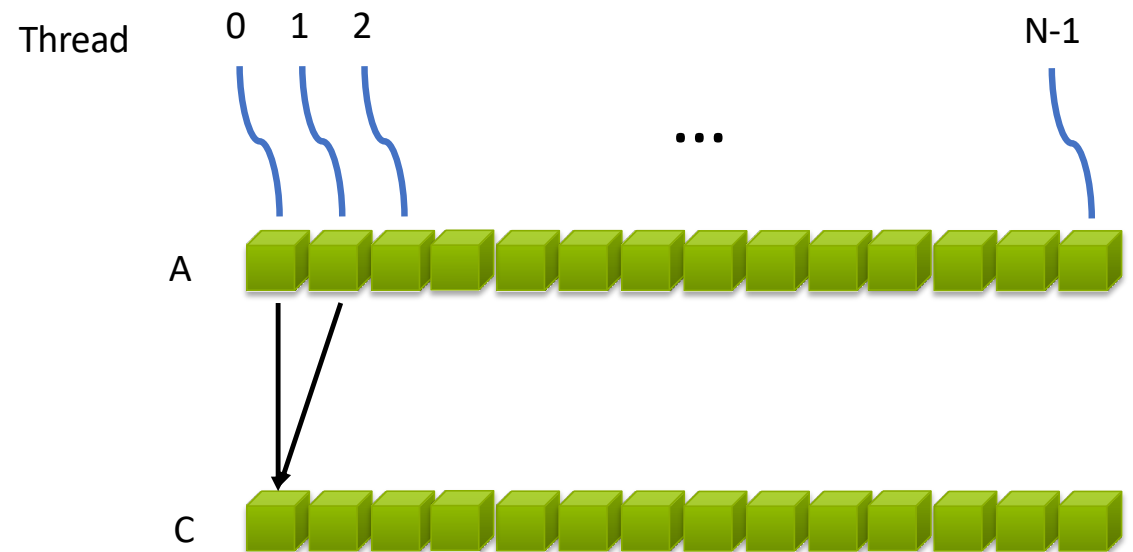
Stencil (1D)

$$A[i-1] + A[i] + A[i+1] = C[i]$$



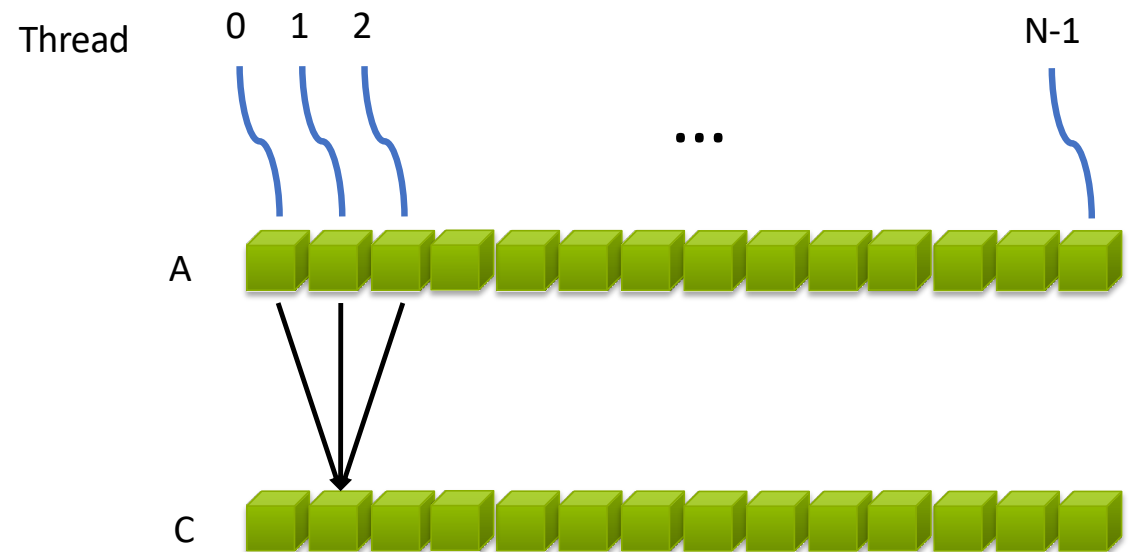
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



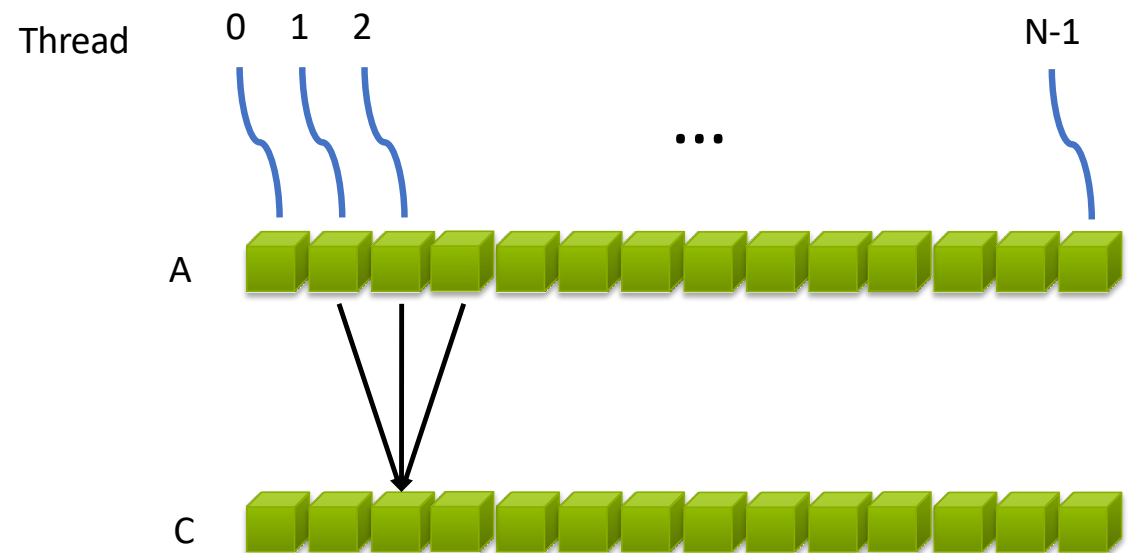
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



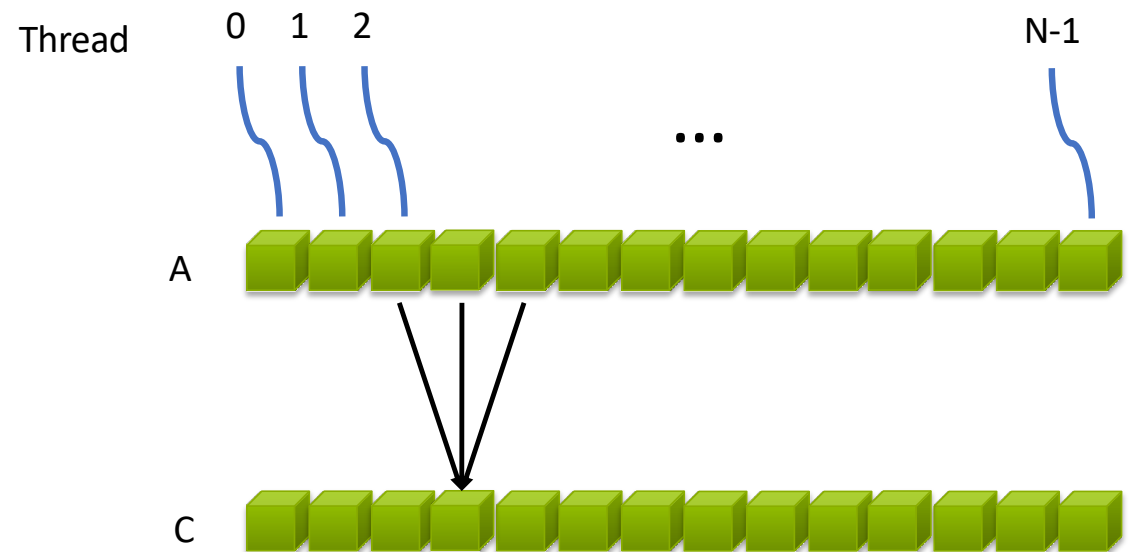
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



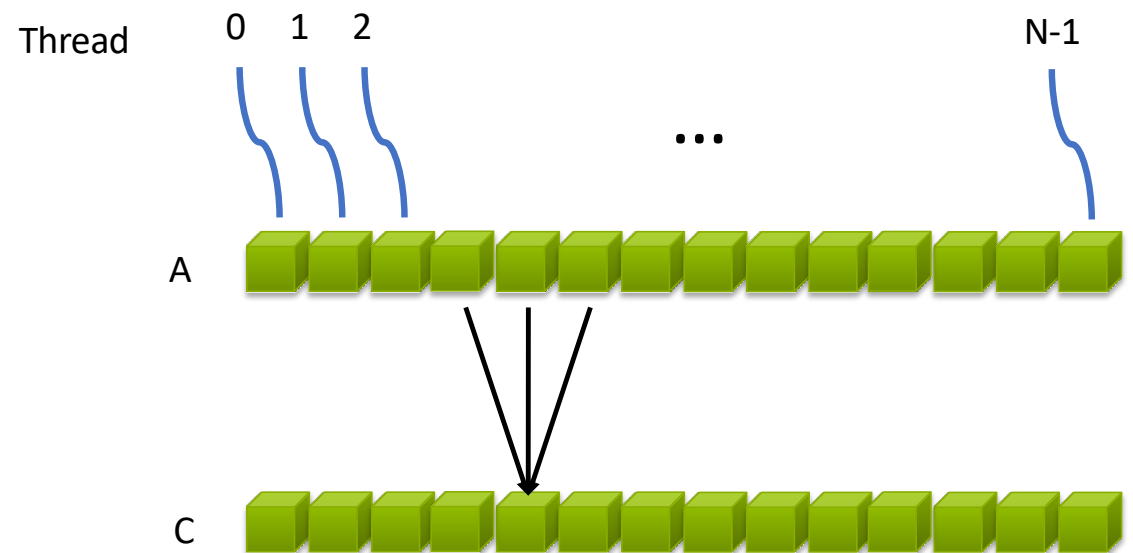
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



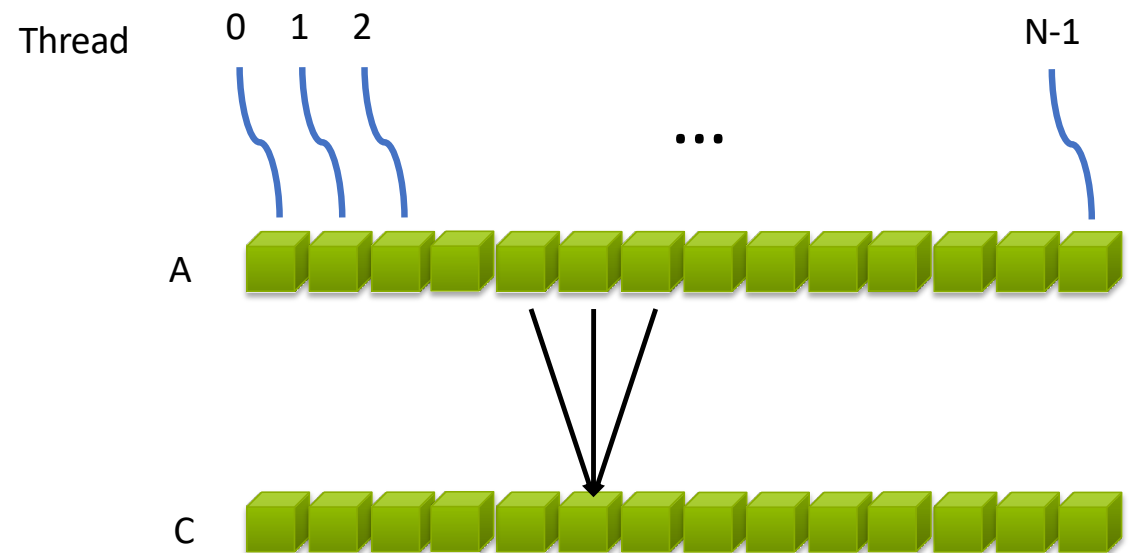
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



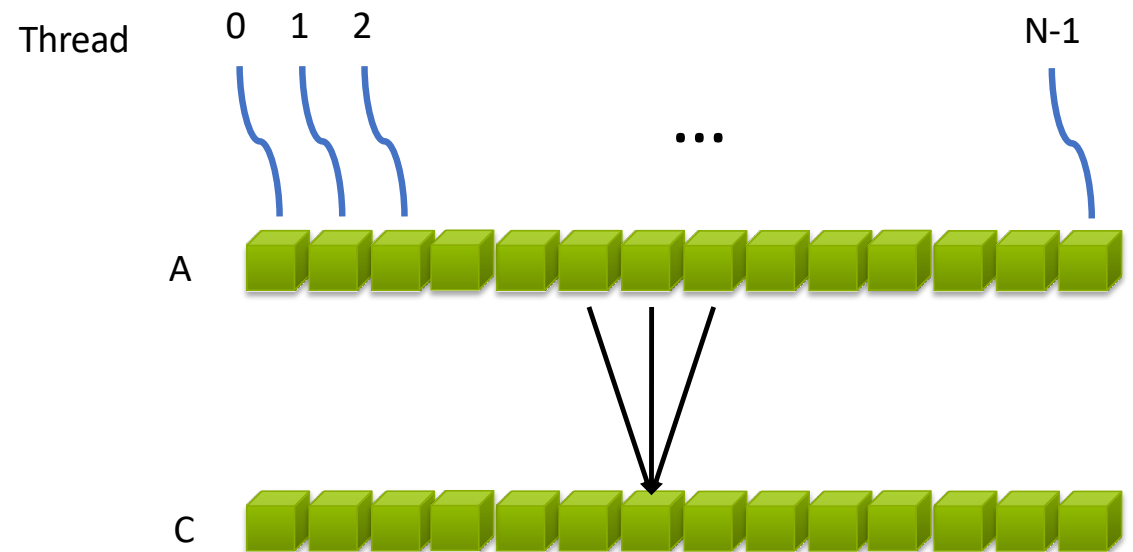
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



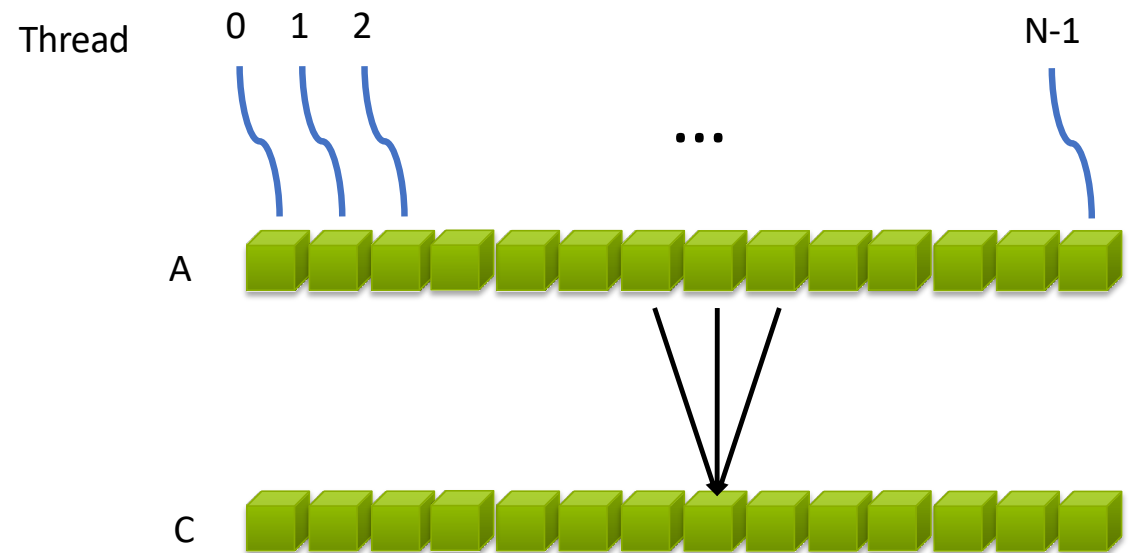
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



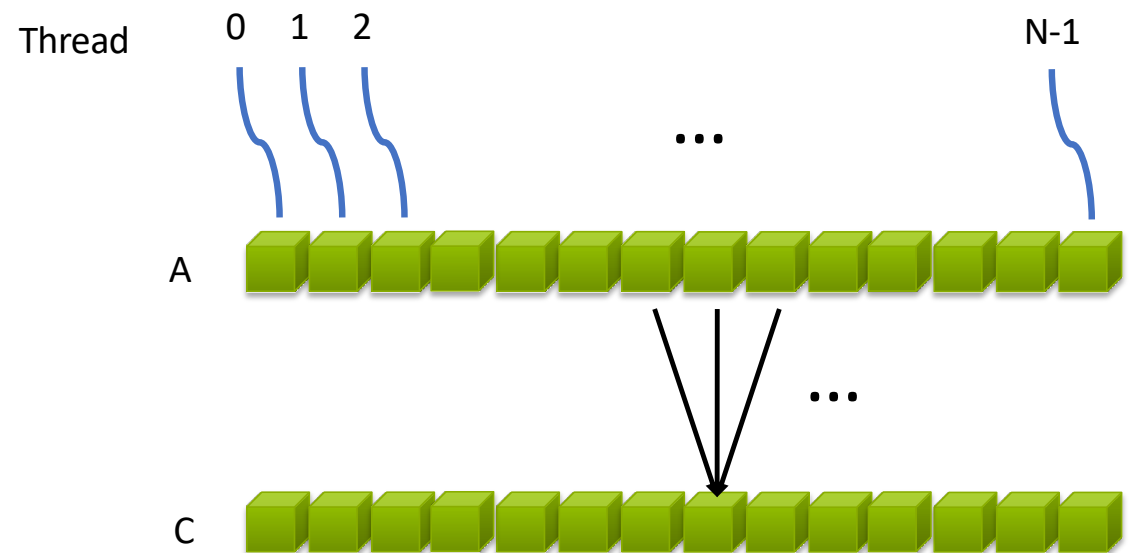
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



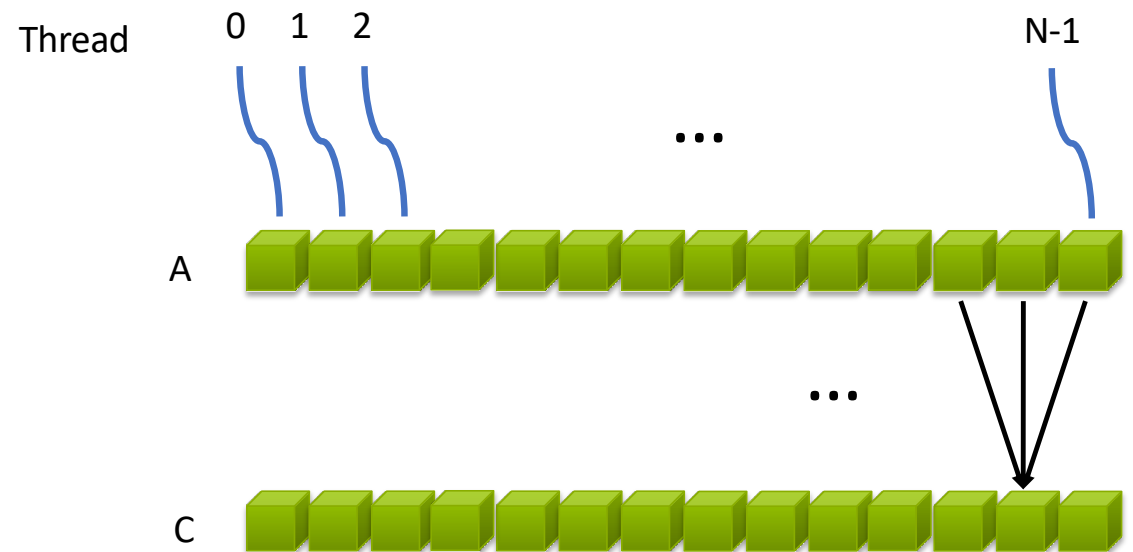
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



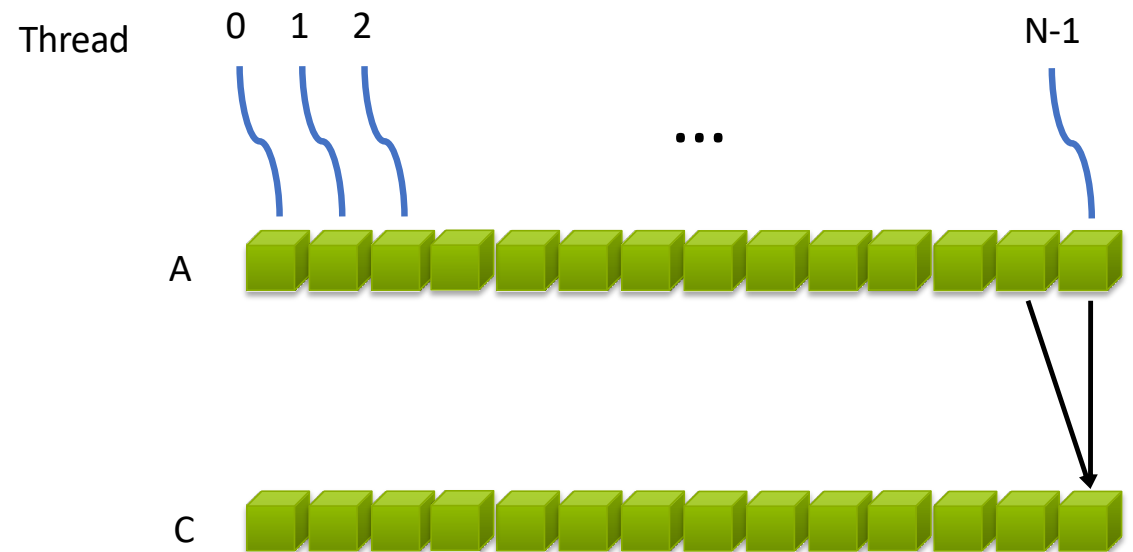
Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$



Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$

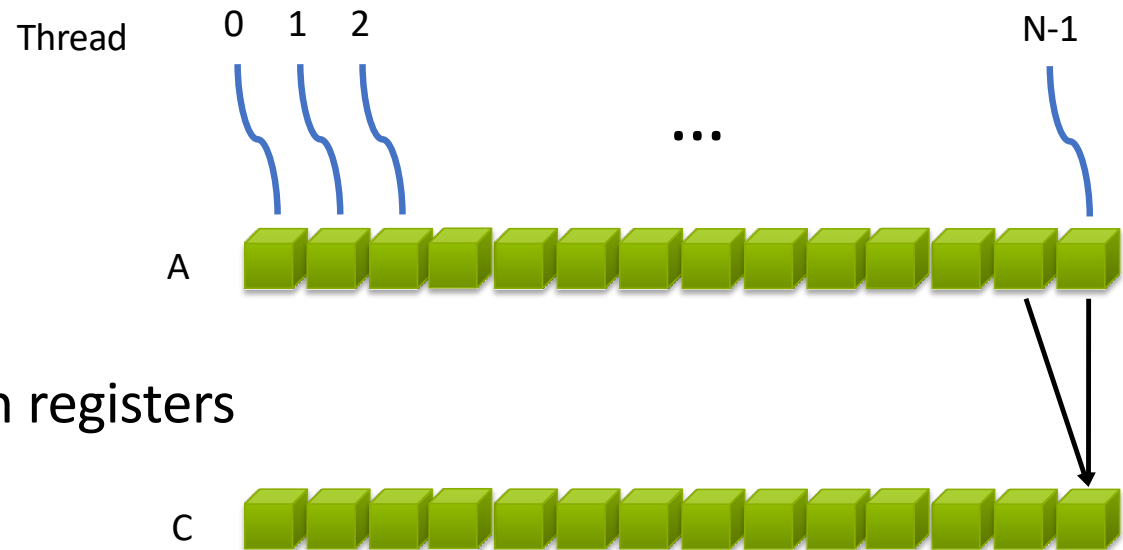


Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$

Each thread:

1. Load $A[i-1]$ to register
2. Load $A[i]$ to register
3. Load $A[i+1]$ to register
4. Compute $A[i-1] + A[i] + A[i+1]$ in registers
5. Store results to $C[i]$



Total amount of data: N input + N output

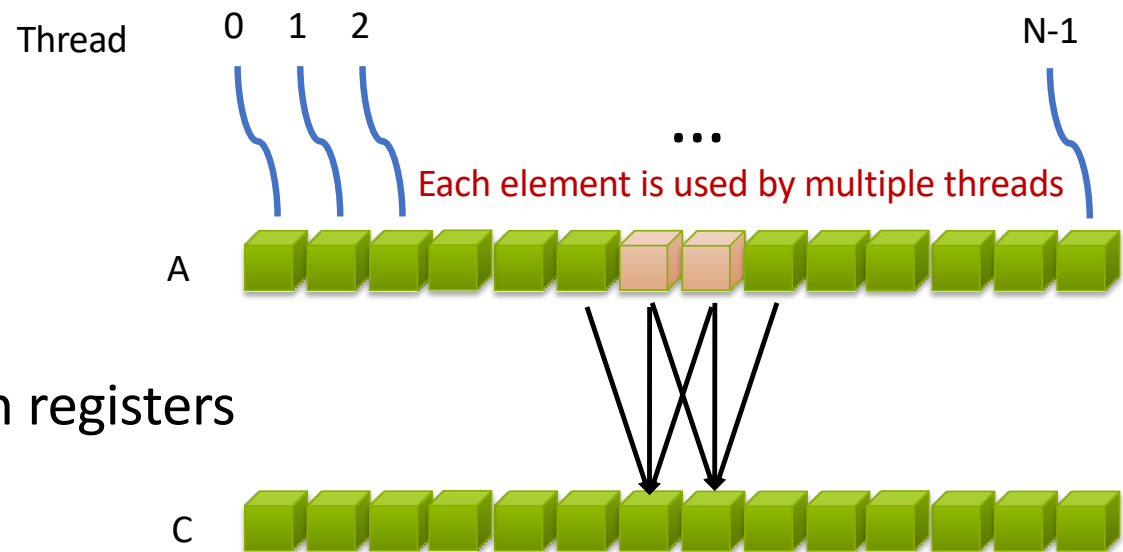
Total memory cost: $3N$ loads + N stores

Stencil

$$A[i-1] + A[i] + A[i+1] = C[i]$$

Each thread:

1. Load $A[i-1]$ to register
2. Load $A[i]$ to register
3. Load $A[i+1]$ to register
4. Compute $A[i-1] + A[i] + A[i+1]$ in registers
5. Store results to $C[i]$

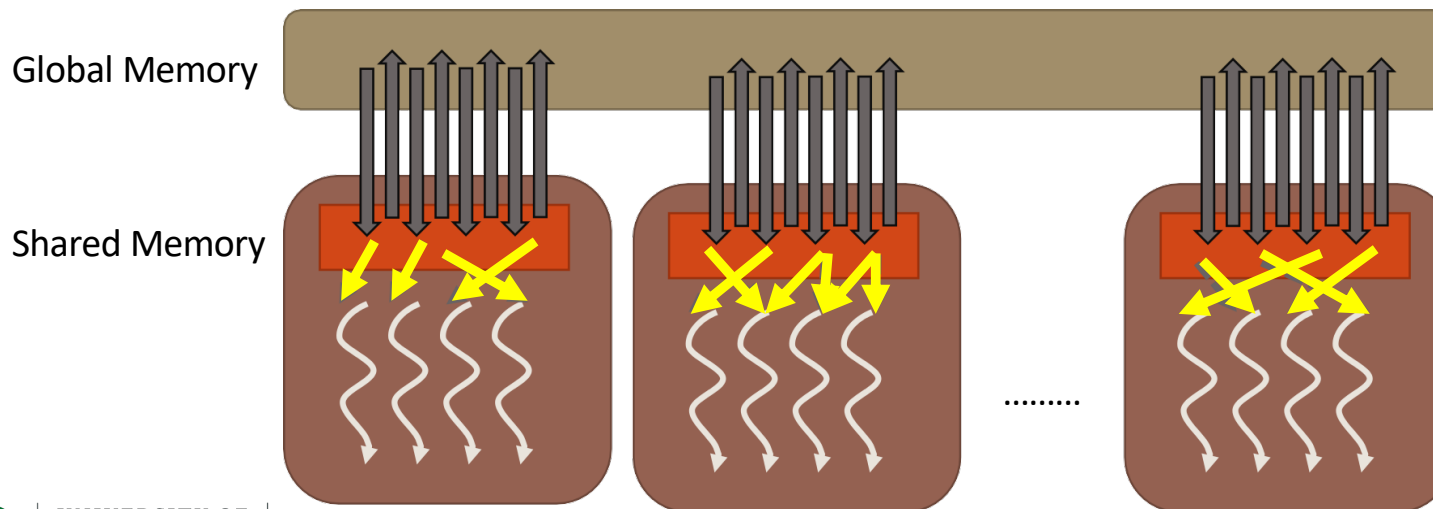


Total amount of data: N input + N output

Total memory cost: $3N$ loads + N stores

Block processing model

- ❑ Partition data into groups that fit into shared memory
- ❑ Load subset of data into shared memory
- ❑ Perform computation on the subset
- ❑ Copy subset back to global memory



Load, execute, and store

Thread processing model

- ❑ From **per thread** view
 - ❑ **Load:** Data from **device** memory to **registers**
 - ❑ **Execute:** Instructions
 - ❑ **Store:** Data back from **registers** to **device** memory

Block processing model

- ❑ From **per block** View
 - ❑ **Load:** Data from **device** memory to **shared** memory
 - ❑ **Execute:** from **per thread** view
 - ❑ **Load:** Data from **shared** memory to **registers**
 - ❑ **Execute:** Instructions
 - ❑ **Store:** Data back from **registers** to **shared** memory
 - ❑ **Store:** Data back from **shared** memory to **device** memory

A Case for Shared Memory

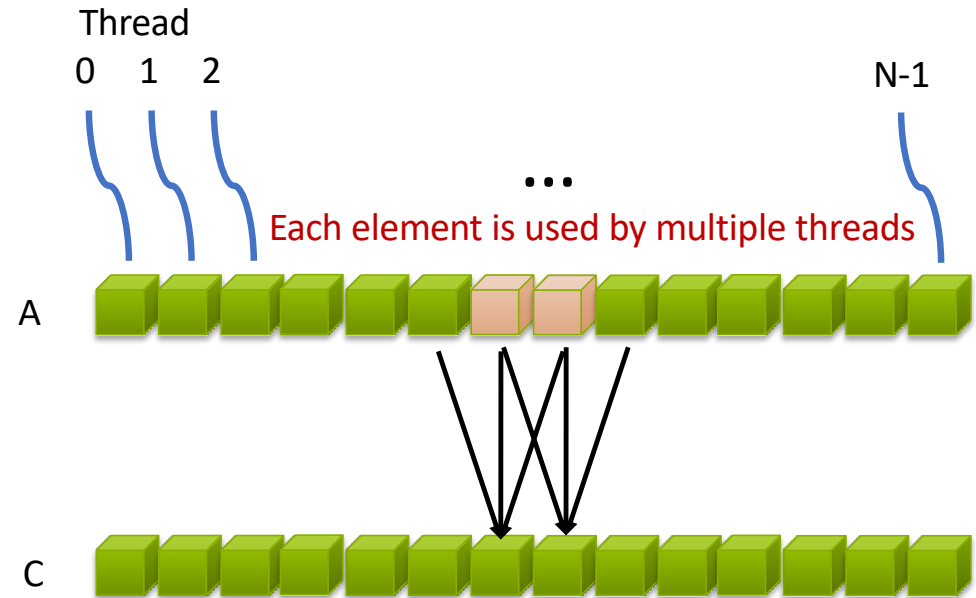
```
__global__ void Stencil_kernel(int *c, int *a)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    int left, right;

    //load value at i-1 left = 0;
    if (i > 0)
        left = a[i - 1];

    //load value at i+1 right = 0;
    if (i < (N - 1))
        right = a[i + 1];

    c[i] = left + a[i] + right; //sum three values
}
```

$$A[i-1] + A[i] + A[i+1] = C[i]$$



Can we optimize it using block processing model?

A Case for Shared Memory

```
__global__ void Stencil_kernel(int *c, int *a)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    int left, right;

    //load value at i-1
    left = 0;
    if (i > 0)
        left = a[i - 1];

    //load value at i+1
    right = 0;
    if (i < (N - 1))
        right = a[i + 1];

    c[i] = left + a[i] + right; //sum three values
}
```

- ❑ Currently: Thread-local computation
- ❑ Bandwidth limited
 - ❑ Requires three loads per thread (at index $i-1$, i , and $i+1$)
- ❑ Block processing model: load each value only once!



CUDA Shared memory

- ❑ Shared memory between threads in the same block can be defined using `__shared__`
- ❑ Shared variables are only accessible from within device functions
 - ❑ Not accessible in host code
- ❑ Must be careful to avoid race conditions
 - ❑ Multiple threads writing to the same shared memory variable
 - ❑ Results in undefined behavior
 - ❑ Typically write to shared memory using `threadIdx`
 - ❑ Thread level synchronization is available through `__syncthreads()`
 - ❑ Synchronizes threads in the block

```
__shared__ int s_data[BLOCK_SIZE];
```



Example

```
__global__ void sum3_kernel(int *c, int *a)
{
    __shared__ int s_data[BLOCK_SIZE];

    int i = blockIdx.x*blockDim.x + threadIdx.x;
    int left, right;

    s_data[threadIdx.x] = a[i];
    __syncthreads();

    //load value at i-1
    left = 0;
    if (i > 0){
        left = s_data[threadIdx.x - 1];
    }

    //load value at i+1
    right = 0;
    if (i < (N - 1)){
        right = s_data[threadIdx.x + 1];
    }

    c[i] = left + s_data[threadIdx.x] + right; //sum
}
```

What is wrong with this code?

- ❑ Allocate a shared array
 - ❑ One integer element per thread
- ❑ Each thread loads a single item to shared memory
- ❑ Call `__syncthreads` to ensure shared memory data is populated by all threads
- ❑ Load all elements through shared memory

Example

```
__global__ void sum3_kernel(int *c, int *a)
{
    __shared__ int s_data[BLOCK_SIZE];

    int i = blockIdx.x*blockDim.x + threadIdx.x;
    int left, right;

    s_data[threadIdx.x] = a[i];
    __syncthreads();

    //load value at i-1
    left = 0;
    if (i > 0){
        if (threadIdx.x > 0)
            left = s_data[threadIdx.x - 1];
        else
            left = a[i - 1];
    }

    //load value at i+1
    right = 0;
    if (i < (N - 1)){
        if (threadIdx.x < (BLOCK_SIZE-1))
            right = s_data[threadIdx.x + 1];
        else
            right = a[i + 1];
    }

    c[i] = left + s_data[threadIdx.x] + right; //sum
}
```

CS 610 GPU Programming – Winter 2026

❑ Additional step required!

❑ Check boundary conditions for the edge of the block

What is an alternative way to fix this?

Problems with Shared memory

- ❑ In the example we saw the introduction of boundary conditions
 - ❑ Global loads still present at boundaries
 - ❑ We have introduced divergence in the code (remember the SIMD model)
 - ❑ This is even more prevalent in 2D examples where we *tile* data into shared memory

```
//boundary condition
left = 0;
if (i > 0){
    if (threadIdx.x > 0)
        left = s_data[threadIdx.x - 1];
    else
        left = a[i - 1];
}
```

Dynamically Assigned Shared Memory

- ❑ It is possible to dynamically assign shared memory at runtime.
- ❑ Requires both a host and device modification to code
 - ❑ Device: Must declare shared memory as extern
 - ❑ Host: Must declare shared memory size in kernel launch parameters

```
unsigned int sm_size = sizeof(float)*n;
image_kernel<<<blocksPerGrid, threadsPerBlock, sm_size >>>(d_image);

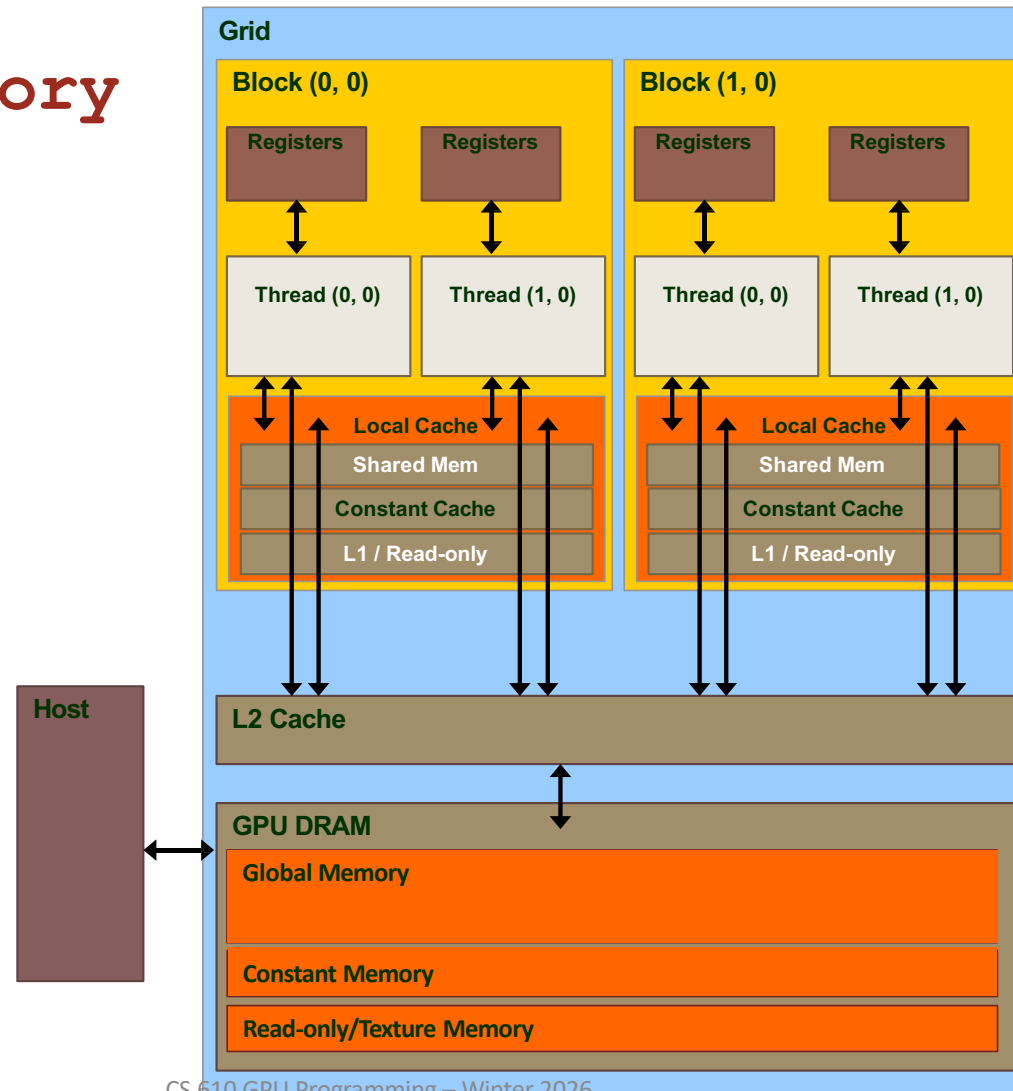
__global__ void image_kernel(float *image)
{
    extern __shared__ float s_data[];
}
```



Constant Memory



Constant Memory



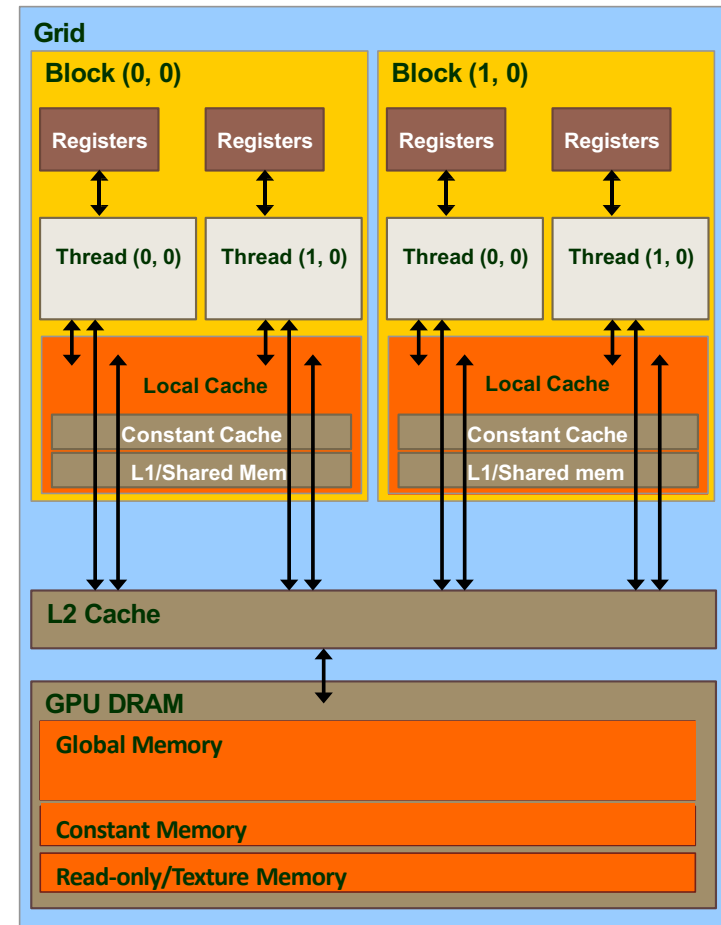
Constant Memory

❑ Constant Memory

- ❑ Stored in the global memory (DRAM)
- ❑ Read through the per SM constant cache (not L1/L2 cache)
- ❑ Writeable by host, read-only for device

❑ How to use

- ❑ Must be **statically** (compile-time) defined as a symbol using `__constant__` qualifier
- ❑ Value(s) must be copied using `cudaMemcpyToSymbol`.



Read-only

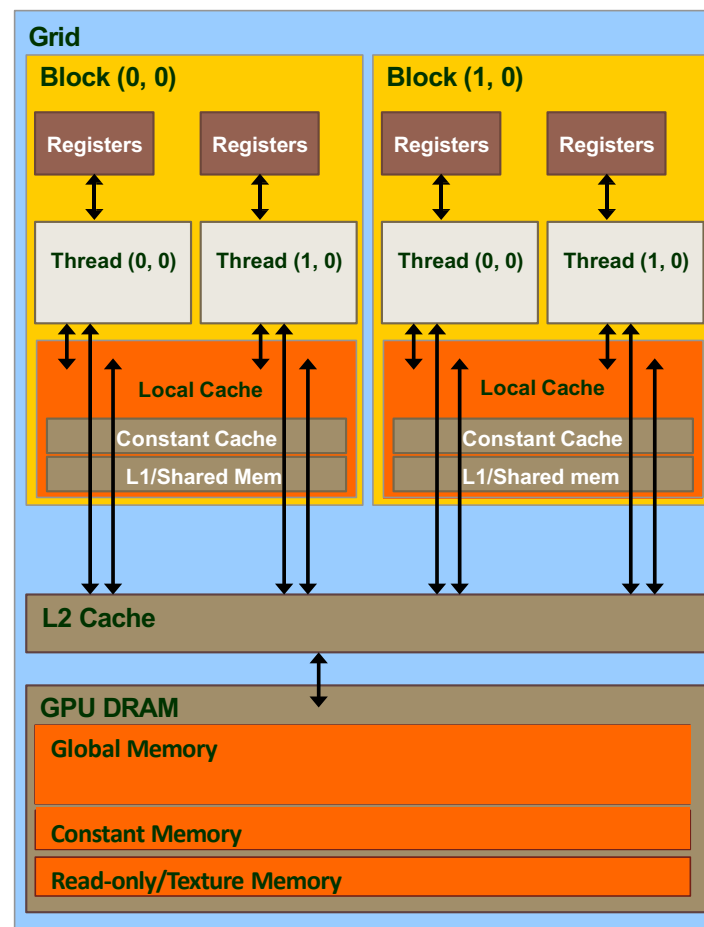
```
_constant_int my_const[16];

__global__ void constant_test() {
    // my_const is read-only on device
}

int main() {
    // my_const is writable on host
    int data[16];
    cudaMemcpySymbol(my_const, data, sizeof(int) * 16);
}
```

Why using constant memory?

- ❑ Aggressively cached in constant cache
 - ❑ Very fast when cache hit
 - ❑ Very slow when no cache hit
- ❑ Constant cache is designed without supporting write operation
 - ❑ much simpler design with high efficiency
 - ❑ Can be as fast as reading from register



When to use constant memory?

- Storing heavily used constant variables
- Size of constant variables are small: less than 64KB
 - 64KB is the constant memory size (allocated in DRAM)
 - Not the size of content cache (unknown)
- Broadcast optimization

Constant Memory Broadcast Optimization

❑.... When values are **broadcast** to threads in a warp

```
__constant__ int my_const[32];

__global__ void vectorAdd() {
    int i = blockIdx.x;

    int value = my_const[i];
}
```

```
__constant__ int my_const[32];

__global__ void vectorAdd() {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    int value = my_const[i];
}
```

Which is good use of constant memory?

Constant Memory Broadcast

```
__constant__ int my_const[32];

__global__ void constant_test() {
    int i = blockIdx.x;

    int value = my_const[i];
}
```

```
__constant__ int my_const[32];

__global__ void constant_test() {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    int value = my_const[i];
}
```

Which is good use of constant memory?

- ☐ Best possible use of constant memory
- ☐ Every thread in a warp reads the same
 - ☐ Index based on `blockIdx`
- ☐ No serialization
 - ☐ 1 read request for every thread!
- ☐ Other threads in the block will also hit cache

- ☐ Worst possible use of constant memory
- ☐ Every thread in a warp reads different value
 - ☐ Index based on `threadIdx`
- ☐ Each access will be serialized
 - ☐ 32 different read requests!
- ☐ Other threads in the block will likely miss the cache

Constant Memory

☐ Question: Should I convert `#define` to constants?

☐ E.g. `#define MY_CONST 1234`

☐ Answer: No

☐ Leave alone

☐ `#defines` are embed in the code by pre-processors

☐ They don't take up registers as they are embedded within the instruction space

☐ i.e. are replaced with literals by the pre-processor

☐ Only replace constants that may change at runtime by host (but not during the GPU programs)



Acknowledgements and Further Reading

- ❑ <http://devblogs.nvidia.com/parallelforall/cuda-pro-tip-kepler-texture-objects-improve-performance-and-flexibility/>
- ❑ Mike Giles (Oxford): Different Memory and Variable Types
 - ❑ <https://people.maths.ox.ac.uk/gilesm/cuda/>
- ❑ CUDA Programming Guide
 - ❑ <http://docs.nvidia.com/cuda/cuda-c-programming-guide/#texture-memory>

Summary of Variables

Variable declaration	Memory	Scope	Lifetime	Performance penalty	Access
<code>int localVar;</code>	register OR device memory (if register spill)	thread	thread	1X or 100X	W/R
<code>Int localArray[10];</code>	register OR device memory (if register spill)	thread	thread	1X or 100X	W/R
<code>__shared__ int sharedVar;</code>	shared memory	block	block	20X	W/R
<code>__device__ int globalVar;</code>	device memory	grid	program	100X	W/R
<code>__constant__ int constantVar;</code>	constant cache	grid	program	1X	R